Trivance: Latency-Optimal AllReduce by Shortcutting Multiport Networks

Anton Juerss[13], Vamsi Addanki[2], Stefan Schmid[13]

1 weizenbaum institut

2 PURDUE UNIVERSITY

3 Technische Universität Berlin

AllReduce and torus networks

AllReduce = Aggregation across all nodes — distribute result to all

Small AllReduce operations are increasingly latency-sensitive

SIGCOMM'26: DynamiQ: Accelerating Gradient Synchronization using Compressed Multi-hop All-reduce
Wenchen Han, Shay Vargaftik, Michael Mitzenmacher, Ran Ben Basat

SIGCOMM'26: OptCCL: Scalable Synthesis of Optimal Collective Communication Algorithms
Richard Shapley, Rachit Agarwal, David Shmoys

SIGCOMM'26: Harvest: Adaptive Photonic Switching Schedules for Collective Communication in Scale-up Domains
Mahir Rahman, Samuel Joseph, Nihar Kodkani, Vamsi Addanki et al.

SIGCOMM'26: Theseus: Runtime-Adaptive GPU Collective Communication with Hot-Swappable Schedules
Rui Ding, Xiandong Lu, Jiajun Wang, Xunpeng Liu et al.

SIGCOMM'25: SyCCL: Exploiting Symmetry for Efficient Collective Communication Scheduling
Jiamin Cao, Shangfeng Shi, Jiaqi Gao, Weisen Liu et al.

NSDI'25: OptiReduce: Resilient and Tail-Optimal AllReduce for Distributed Deep Learning in the Cloud
Ertza Warraich, Omer Shabtai, Khalid Manaa, Shay Vargaftik et al.

SIGCOMM'25: ResCCL: Resource-Efficient Scheduling for Collective Communication
Tongrui Liu, Chenyang Hei, Fuliang Li, Chengxi Gao et al.

NSDI'24: Swing: Short-cutting Rings for Higher Bandwidth Allreduce
Daniele De Sensi, Tommaso Bonato, David Saam, Torsten Hoefler

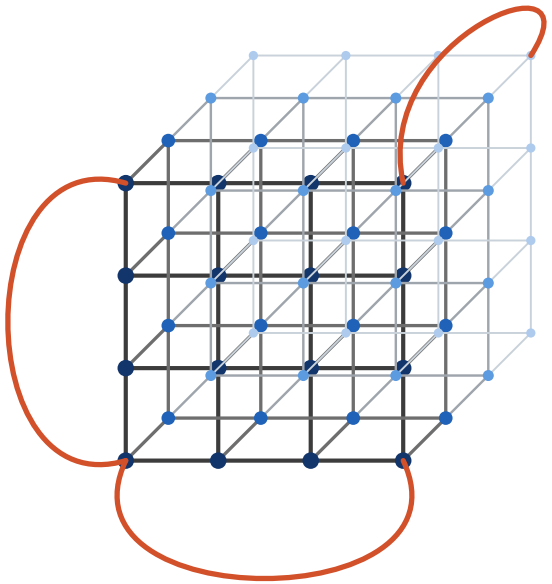
SC'21: Flare: Flexible In-Network Allreduce
Daniele De Sensi, Salvatore Di Girolamo, Saleh Ashkboos, Torsten Hoefler et al.

AllReduce and torus networks

AllReduce = Aggregation across all nodes — distribute result to all

Small AllReduce operations are increasingly latency-sensitive

Torus = mesh/grid with wrap-around links
— every dimension is a ring



— Neighbor links

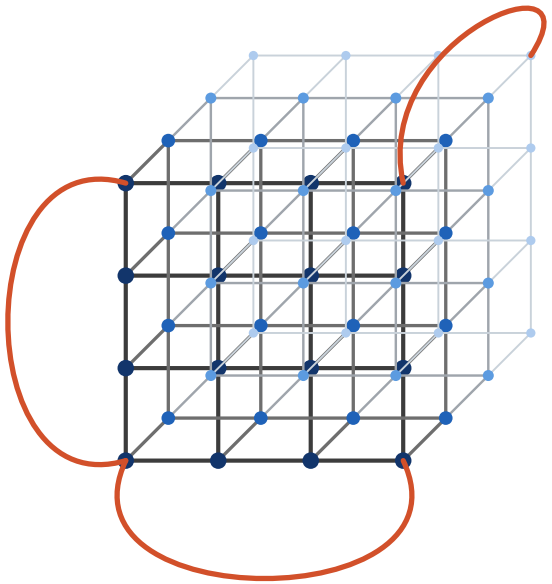
Wrap-around links

AllReduce and torus networks

AllReduce = Aggregation across all nodes — distribute result to all

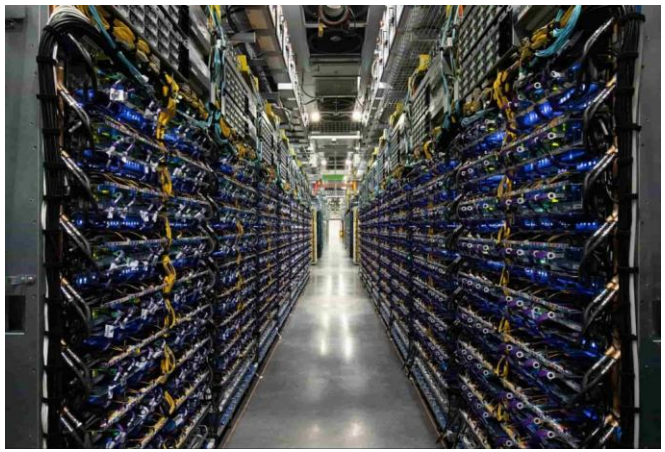
Small AllReduce operations are increasingly latency-sensitive

Torus = mesh/grid with wrap-around links
— every dimension is a ring



— Neighbor links

Wrap-around links



Google TPU Pods

up to 4,096 chips on a 3D torus (TPUv4);
TPU 8t scales a superpod to 9,600

AWS Trainium2 UltraServer

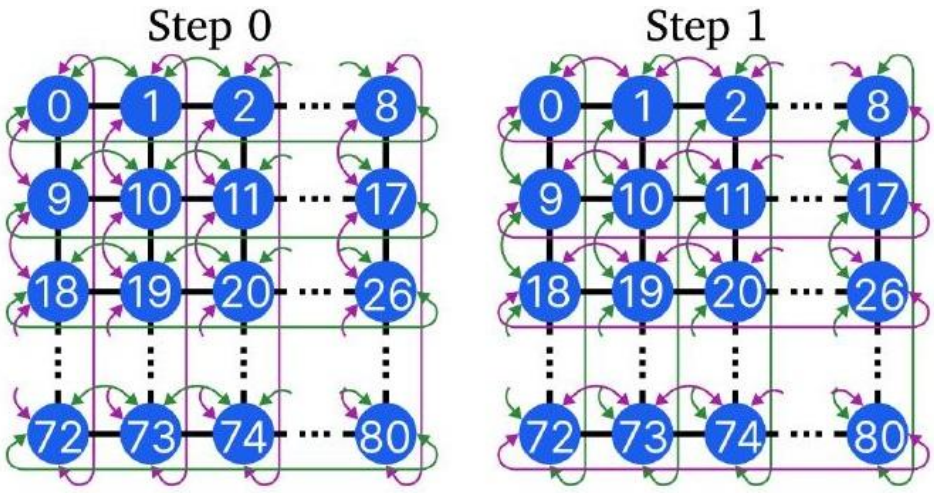
64 chips on a 4×4×4 3D torus

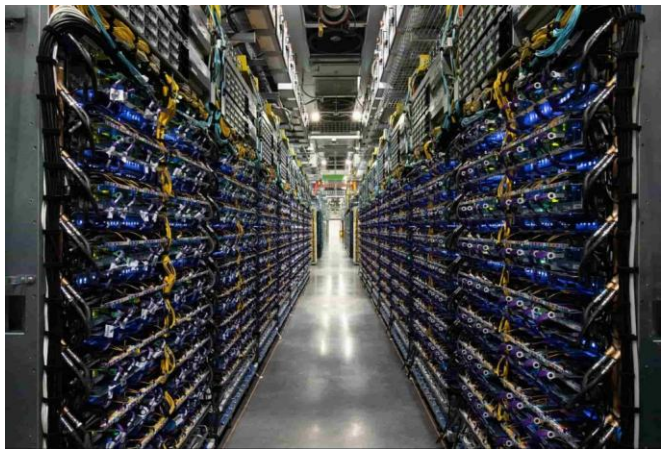
AllReduce and torus networks

AllReduce = Aggregation across all nodes — distribute result to all

Small AllReduce operations are increasingly latency-sensitive

Torus = mesh/grid with wrap-around links
— every dimension is a ring





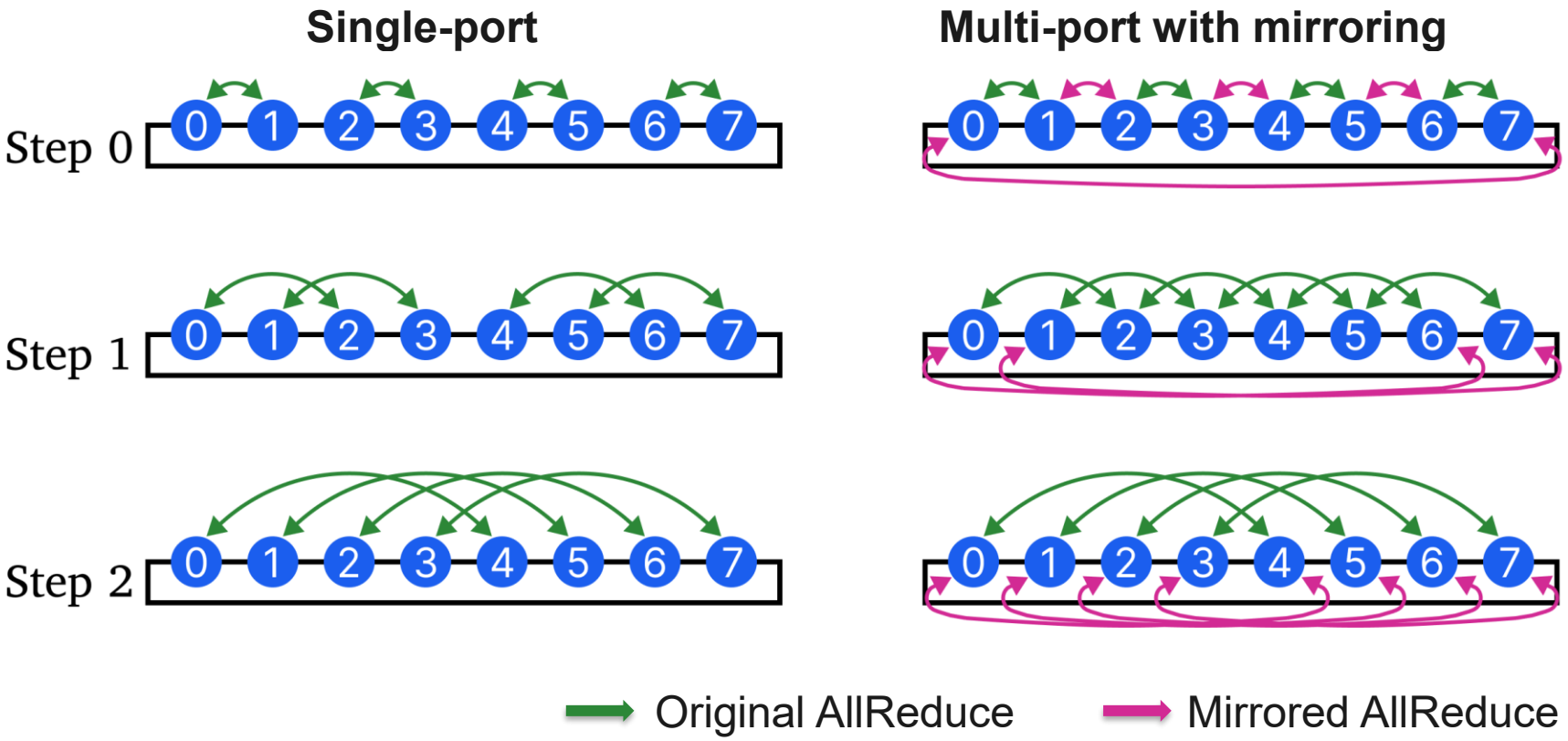
Google TPU Pods

up to 4,096 chips on a 3D torus (TPUv4);
TPU 8t scales a superpod to 9,600

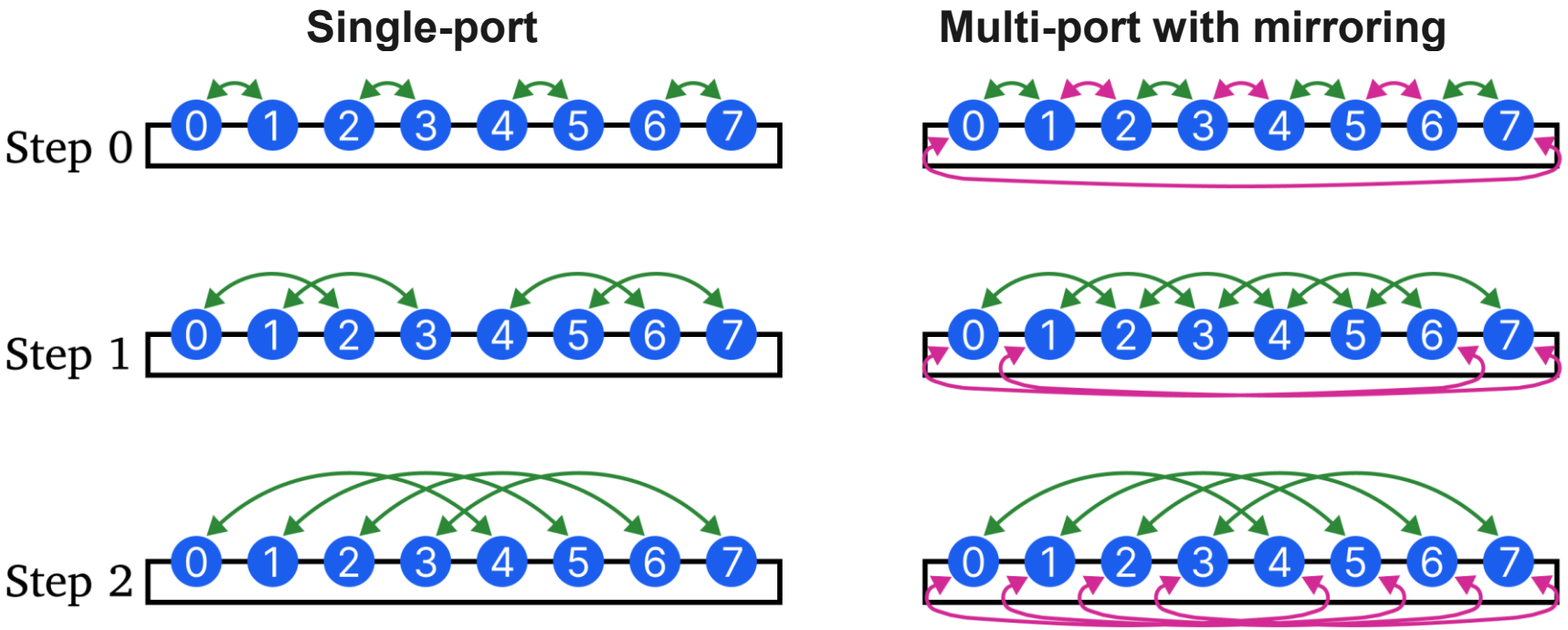
AWS Trainium2 UltraServer

64 chips on a 4×4×4 3D torus

Existing multi-port extension: collective mirroring

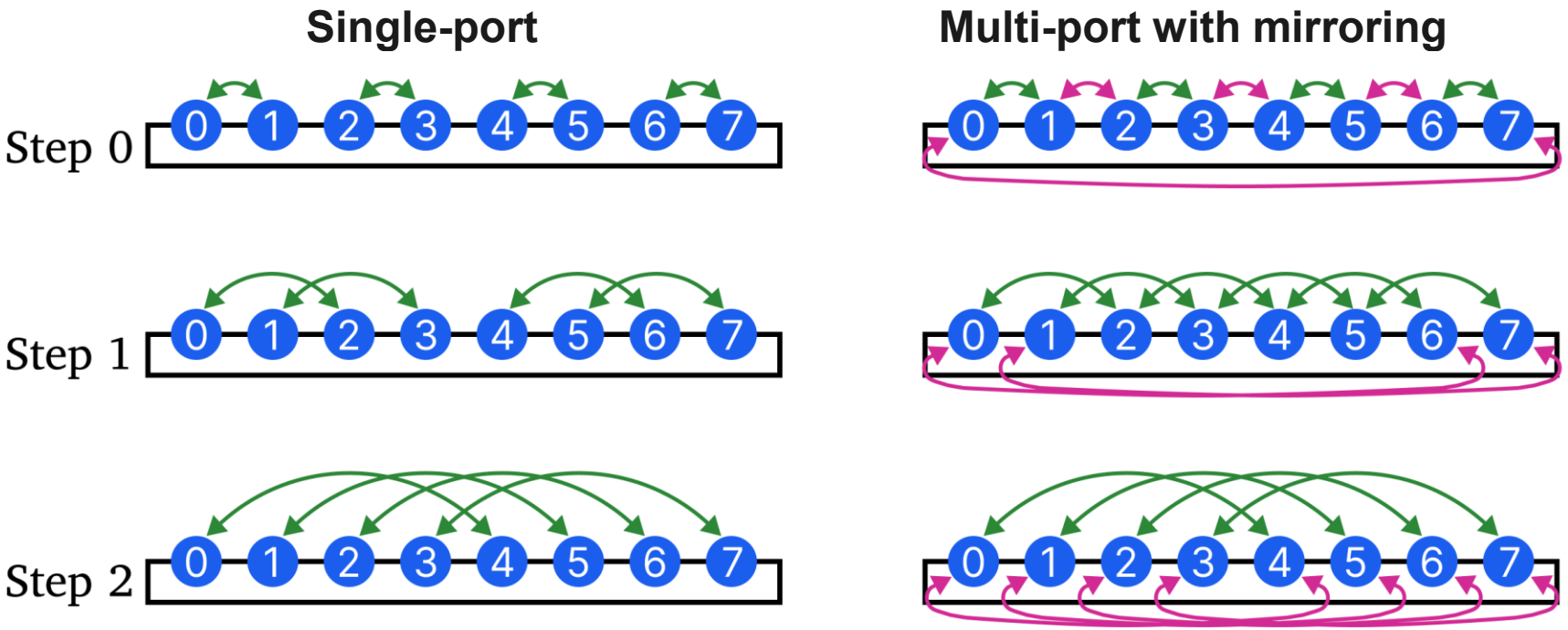


Existing multi-port extension: collective mirroring



Result: available bandwidth doubles — latency remains identical

Existing multi-port extension: collective mirroring



Result: available bandwidth doubles — latency remains identical

Research Questions: ***Can both ports serve the same collective?***
Reduce the number of communication steps?
Minimize the communication distance?

Trivance

↔ **Use BOTH ports simultaneously**
Communicate left and right with the same distance offset

⊕ **Joint reduction**
Both incoming messages are reduced together

Trivance

↔ **Use BOTH ports simultaneously**
Communicate left and right with the same distance offset

×3 **Triple communication distance**
Distance equals 3^k per step k

⊕ **Joint reduction**
Both incoming messages are reduced together

∧ **Latency-optimal**
Trivance reaches all nodes in $\log_3 n$ steps

Trivance

↔ **Use BOTH ports simultaneously**
Communicate left and right with the same distance offset

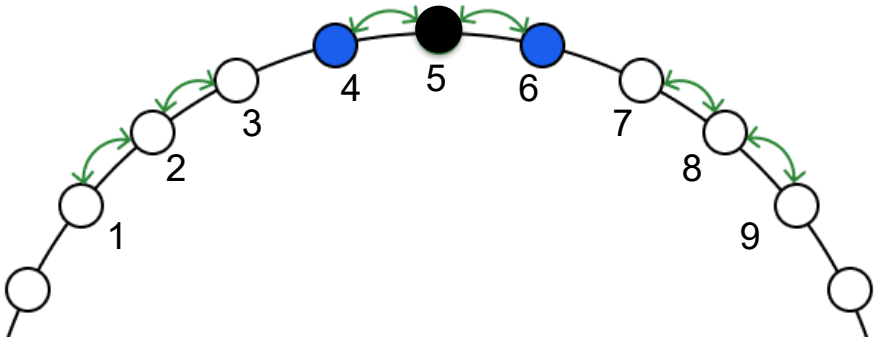
⊕ **Joint reduction**
Both incoming messages are reduced together

×3 **Triple communication distance**
Distance equals 3^k per step k

∧ **Latency-optimal**
Trivance reaches all nodes in $\log_3 n$ steps

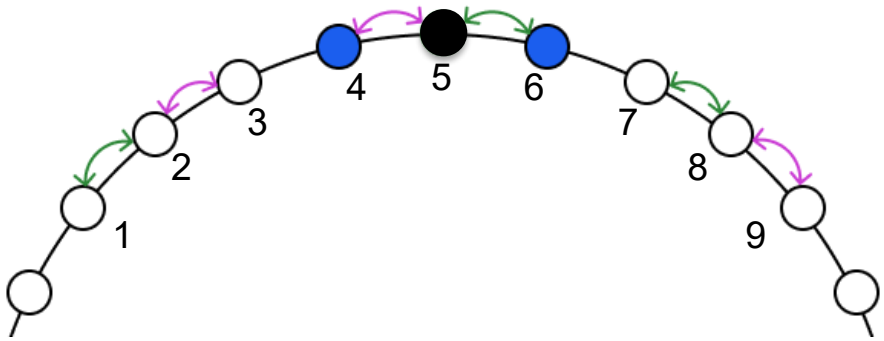
Step $k = 0$

Trivance





Recursive Doubling





Trivance

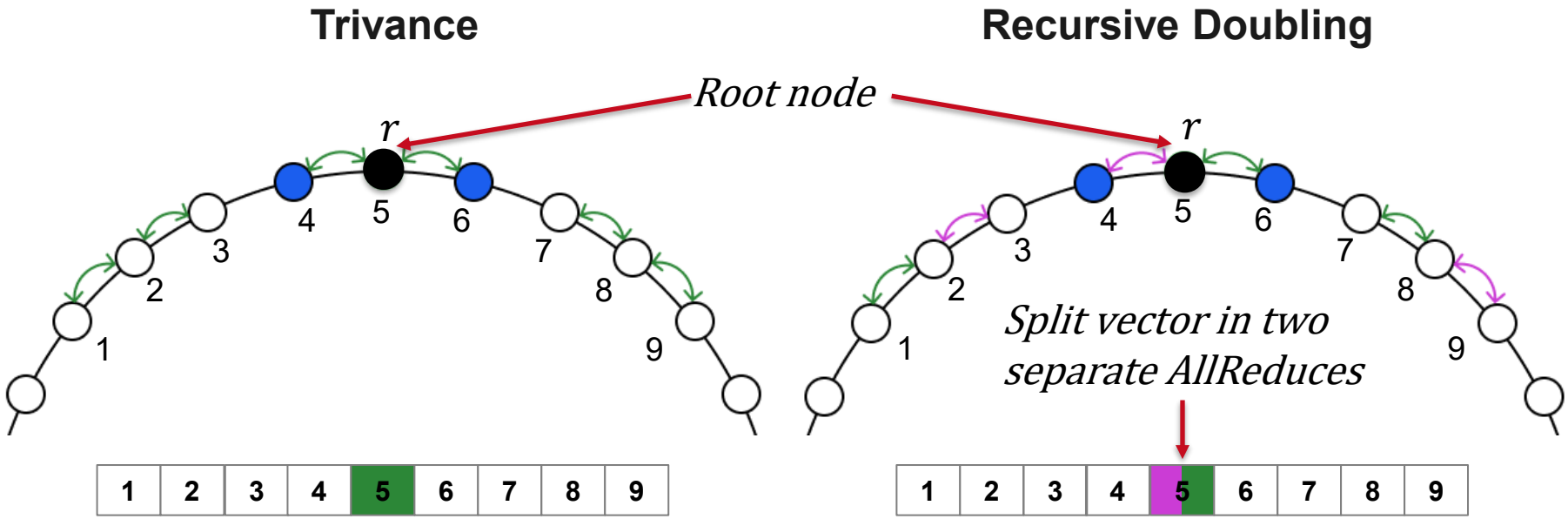
↔ **Use BOTH ports simultaneously**
Communicate left and right with the same distance offset

⊕ **Joint reduction**
Both incoming messages are reduced together

×3 **Triple communication distance**
Distance equals 3^k per step k

∧ **Latency-optimal**
Trivance reaches all nodes in $\log_3 n$ steps

Step $k = 0$



Trivance

↔ **Use BOTH ports simultaneously**
Communicate left and right with the same distance offset

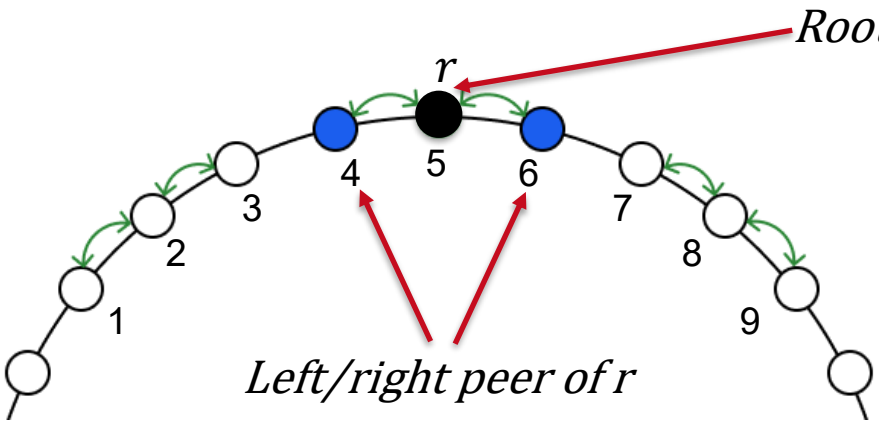
⊕ **Joint reduction**
Both incoming messages are reduced together

×3 **Triple communication distance**
Distance equals 3^k per step k

∧ **Latency-optimal**
Trivance reaches all nodes in $\log_3 n$ steps

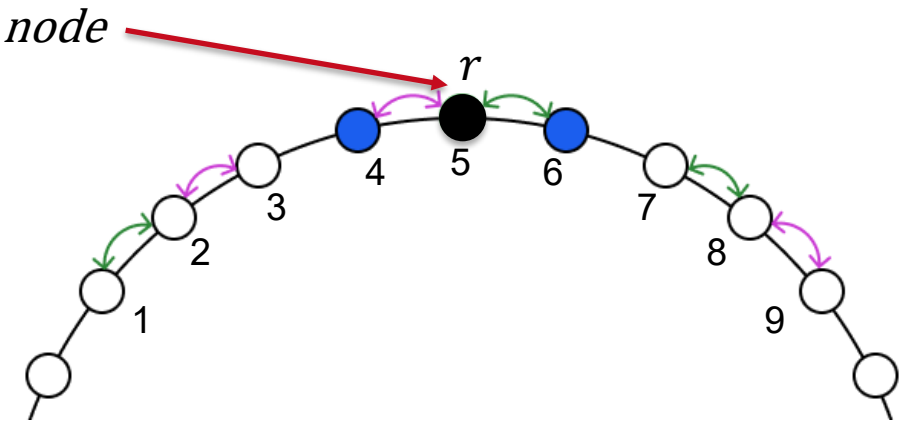
Step $k = 0$

Trivance





Recursive Doubling





Trivance

↔ **Use BOTH ports simultaneously**
Communicate left and right with the same distance offset

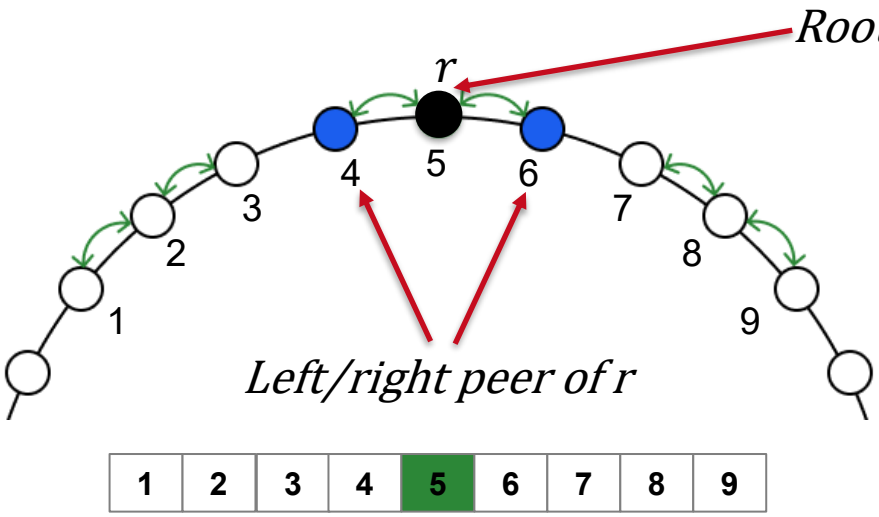
⊕ **Joint reduction**
Both incoming messages are reduced together

×3 **Triple communication distance**
Distance equals 3^k per step k

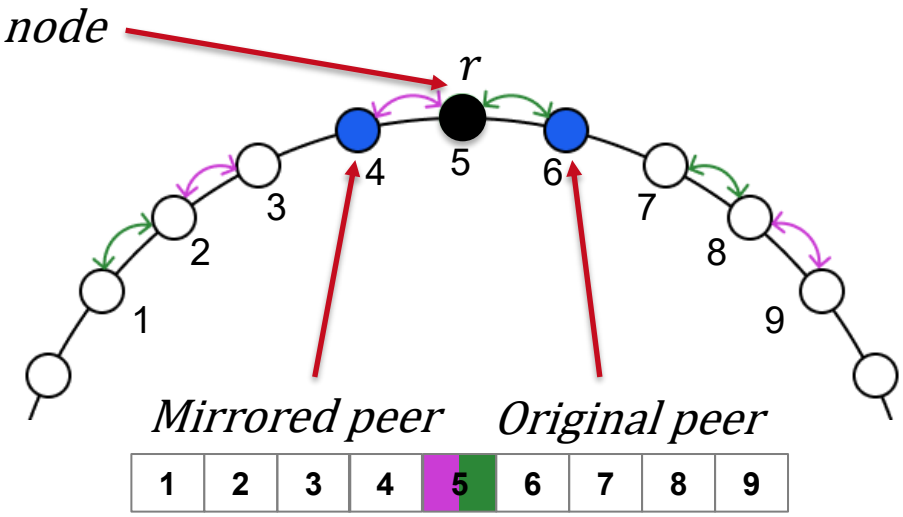
∧ **Latency-optimal**
Trivance reaches all nodes in $\log_3 n$ steps

Step $k = 0$

Trivance



Recursive Doubling



Trivance

↔ **Use BOTH ports simultaneously**
Communicate left and right with the same distance offset

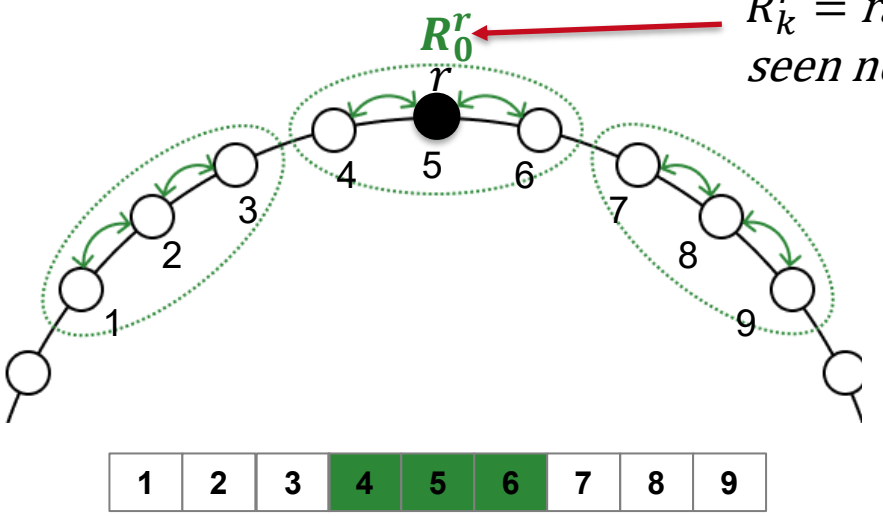
⊕ **Joint reduction**
Both incoming messages are reduced together

×3 **Triple communication distance**
Distance equals 3^k per step k

∧ **Latency-optimal**
Trivance reaches all nodes in $\log_3 n$ steps

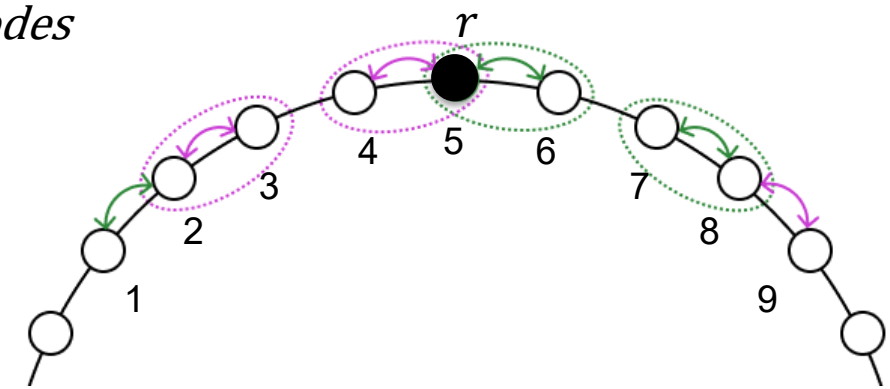
After Step k = 0

Trivance



R_k^r = *range of seen nodes*

Recursive Doubling





Trivance

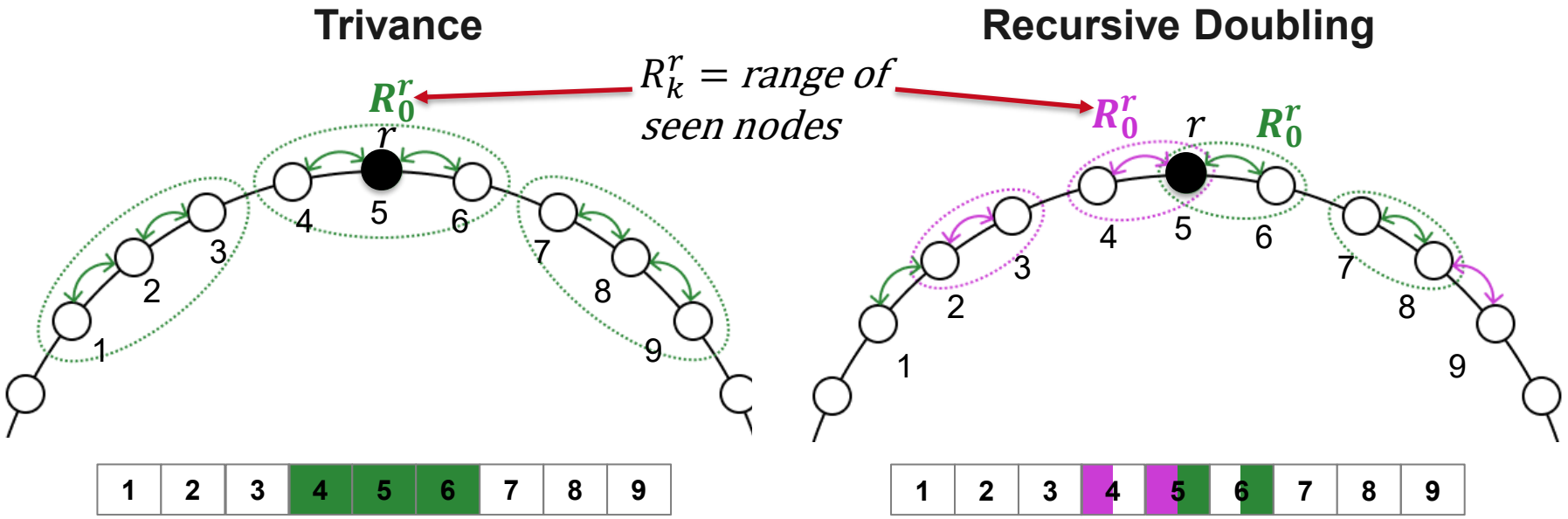
↔ **Use BOTH ports simultaneously**
Communicate left and right with the same distance offset

⊕ **Joint reduction**
Both incoming messages are reduced together

×3 **Triple communication distance**
Distance equals 3^k per step k

∧ **Latency-optimal**
Trivance reaches all nodes in $\log_3 n$ steps

After Step k = 0



Trivance

↔ **Use BOTH ports simultaneously**
Communicate left and right with the same distance offset

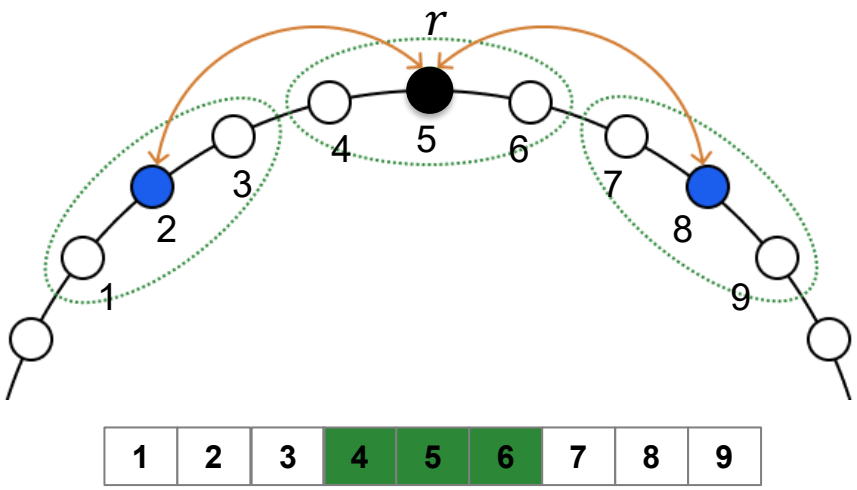
⊕ **Joint reduction**
Both incoming messages are reduced together

×3 **Triple communication distance**
Distance equals 3^k per step k

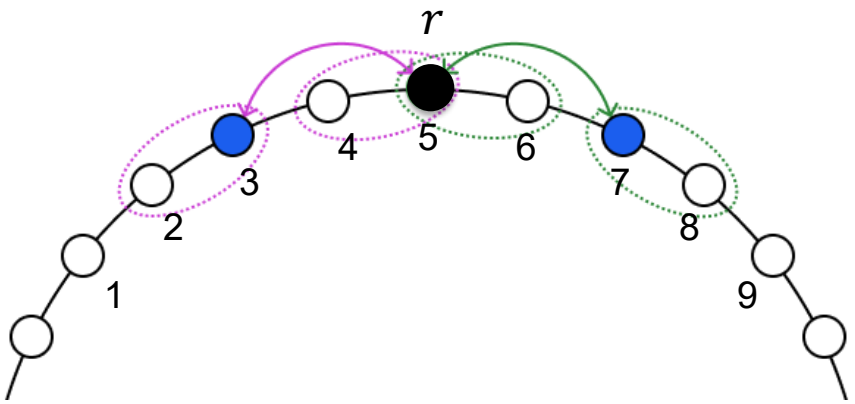
∧ **Latency-optimal**
Trivance reaches all nodes in $\log_3 n$ steps

Step $k = 1$

Trivance



Recursive Doubling





Trivance

↔ **Use BOTH ports simultaneously**
Communicate left and right with the same distance offset

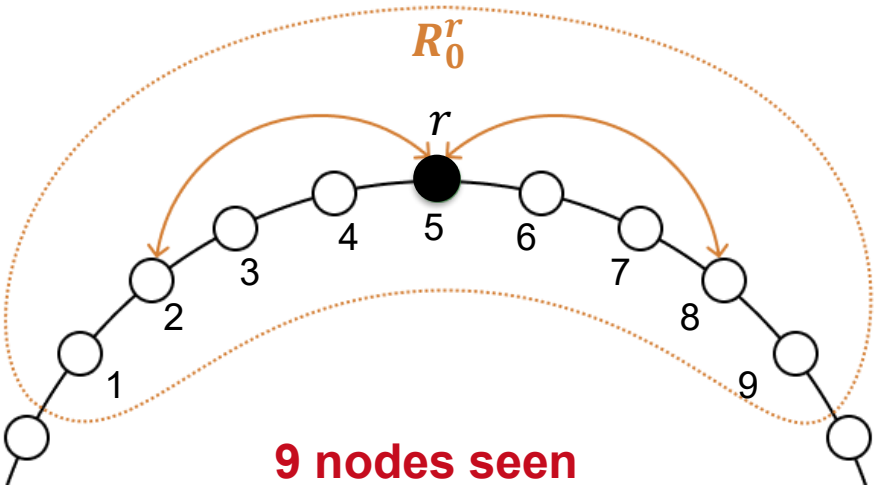
⊕ **Joint reduction**
Both incoming messages are reduced together

×3 **Triple communication distance**
Distance equals 3^k per step k

∧ **Latency-optimal**
Trivance reaches all nodes in $\log_3 n$ steps

After Step $k = 1$

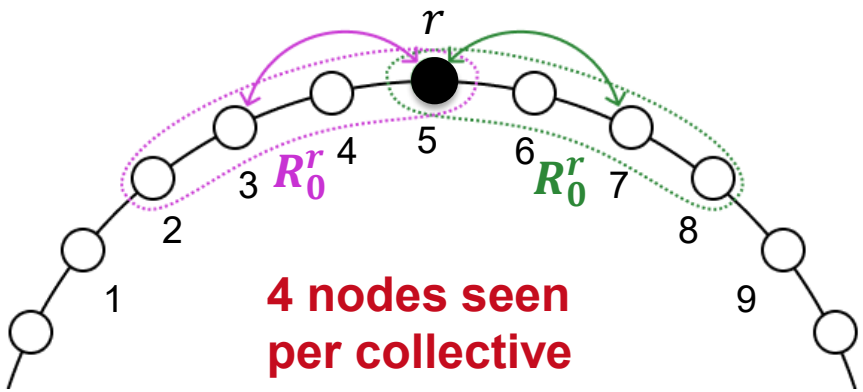
Trivance



9 nodes seen



Recursive Doubling



4 nodes seen per collective



Tradeoffs and optimality factors

	Ring	Rec.Doub.	Swing	Trivance
Communication steps	$2n$	$\log_2 n$	$\log_2 n$	$\log_3 n$

Optimal | Near-Optimal | Average | Worst

Tradeoffs and optimality factors

	Ring	Rec.Doub.	Swing	Trivance
Communication steps	$2n$	$\log_2 n$	$\log_2 n$	$\log_3 n$
Number of hops	$2n$	n	$\frac{2n}{3}$	$\frac{n}{2}$

Optimal | Near-Optimal | Average | Worst

Tradeoffs and optimality factors

	Ring	Rec.Doub.	Swing	Trivance
Communication steps	$2n$	$\log_2 n$	$\log_2 n$	$\log_3 n$
Number of hops	$2n$	n	$\frac{2n}{3}$	$\frac{n}{2}$
Transmission delay	$2m$	$m\log_2 n$	$\frac{2m}{3}\log_2 n$	$\frac{4m}{3}\log_3 n$

Transmitted data size times number of overlapping links

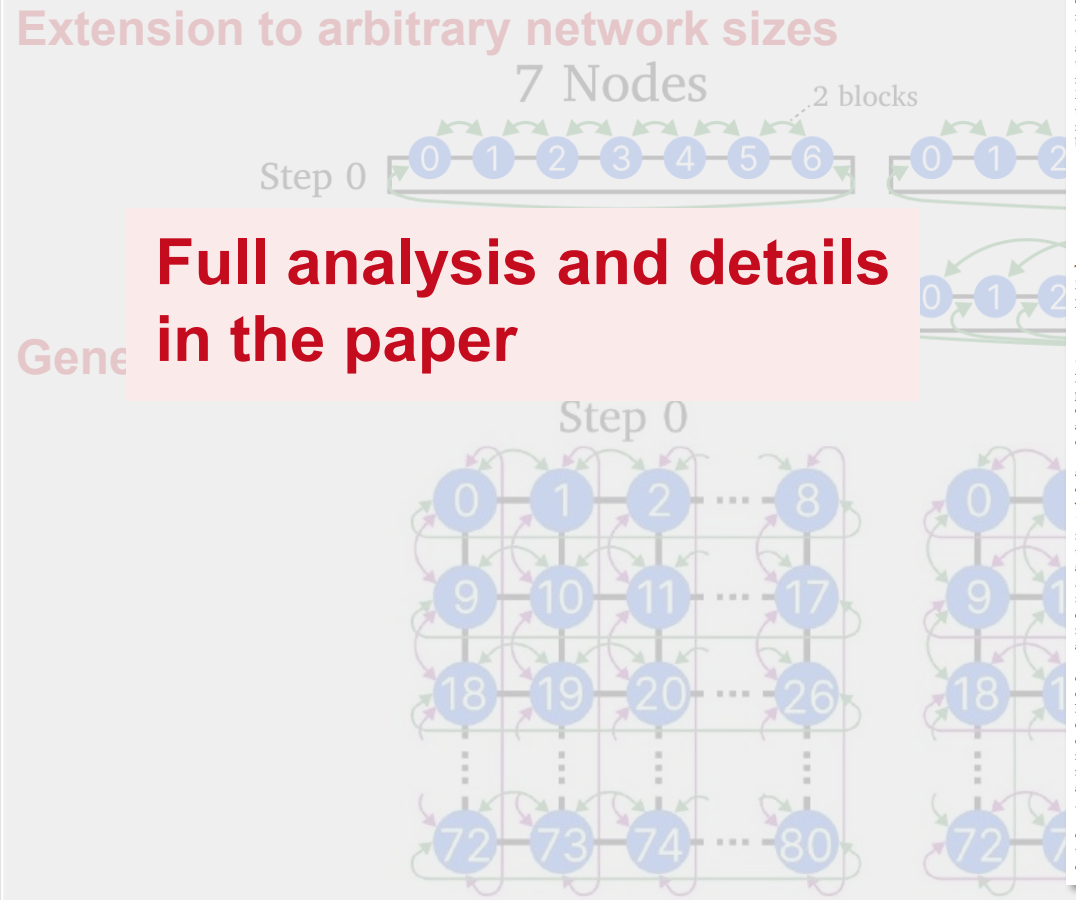
Optimal | Near-Optimal | Average | Worst

Tradeoffs and optimality factors

	Ring	Rec.Doub.	Swing	Trivance
Communication steps	2n	$\log_2 n$	$\log_2 n$	$\log_3 n$
Number of hops	2n	n	$\frac{2n}{3}$	$\frac{n}{2}$
Transmission delay	2m	$m\log_2 n$	$\frac{2m}{3}\log_2 n$	$\frac{4m}{3}\log_3 n$
Transmitted data size times number of overlapping links				
Transmission delay in a 3D torus	2m	2.34m	2.08m	2.16m

Optimal | Near-Optimal | Average | Worst

Generalization to multidimensional tori
Extension to arbitrary network sizes



require $\lceil \log_2 n \rceil$ steps, the asymptotic improvement in communication steps is 58.5%. For power-of-two network sizes, this ranges from 25% to 57%, while for arbitrary network sizes it can reach up to 75%. Blocks are propagated according to Algorithm 1, ensuring that each block is forwarded at most once per phase and that the total transmitted volume remains bounded by $2m$ per node for AllReduce. Furthermore, the transmission delay cost is strictly lower than that of the next larger power-of-three topology, since the final communication step covers only the remaining uncovered nodes. The exact overhead therefore depends on the position of n between two consecutive powers of three.

n	Next 3^k	Final Step Congestion	Relative congestion
8	9	3	100.0%
16	27	4	44.4%
32	81	3	11.1%
64	81	19	70.4%

Table 2: Final-step congestion for arbitrary-size TRIVANCE's last step relative to the last step congestion of the next larger power-of-three network size.

5 Multidimensional Tori

In Section 4, we introduced the novel bidirectional communication pattern TRIVANCE for the ring topology. This concept naturally generalizes to higher-dimensional torus networks. Similar to existing approaches [15, 28, 29] TRIVANCE performs one collective for each of the D dimensions with $\frac{1}{D}$ of the initial data vector of size m.

Figure 5 illustrates the communication pattern of TRIVANCE on a square torus of size $n = 81$. In step 0, the original and mirrored collectives communicate at distance one along the horizontal and vertical dimensions, respectively. They switch dimensions in step 1 and return to their initial dimensions in step 2. Each collective is mapped to a distinct dimension in every step, ensuring that their communications remain disjoint and do not interfere. This allows the transmission of each node's initial data vector of size m to be split across the additional communication links, reducing the message size per transmission. At the same time, nodes communicate with closer neighbors, which decreases both communication distance and congestion. This applies to both the latency- and bandwidth-optimal version of TRIVANCE.

Running AllReduce on multidimensional torus networks has no effect on the per step latency of all algorithms. The total transmitted data, and thus the bandwidth-optimality, remains also unchanged. In contrast, multidimensional torus networks of size n have a significant impact on the transmission delay optimality compared to rings of the same size. Table 3 presents the transmission delay optimality for TRIVANCE and state-of-the-art algorithms. Since the closed-form expressions are hardly readable for the bandwidth-optimal algorithms, we also provide the asymptotic optimality values for $D = 2, 3, 4$ as $n \to \infty$.

While the Bucket algorithm achieves optimal performance, the other bandwidth-optimal algorithms converge to an optimality factor of 1 as the dimension size increases. As mentioned prior, this emerges from reducing per-message size due to increased maximum

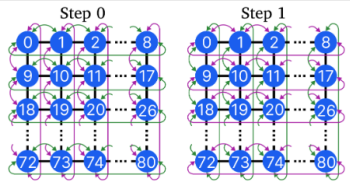


Figure 5: First two steps of TRIVANCE AllReduce for a square torus of size $n = 81$. Green arrows show the original collective; purple the mirrored collective.

injection bandwidth and avoiding congestion through shorter communication distances. For example, with $D = 2$, Swing incurs 20% higher cost than the optimum, and TRIVANCE 33%. As the dimensionality increases, this limitation diminishes for TRIVANCE, dropping to 2% for $D = 4$, almost reaching the optimal transmission delay; Recursive Doubling (B) differs by 7%. This shows that TRIVANCE theoretically can match bandwidth-optimized approaches like Swing and Bucket in transmission delay optimality, while preserving its 50% latency improvement.

Algorithm	Closed-form	D=2	D=3	D=4
Rec.Doub. (L)	$D^2 \sqrt[D]{n}$	$4\sqrt{n}$	$9n^{1/3}$	$16n^{1/4}$
Swing (L)	$\frac{D^2}{3} \sqrt[D]{n}$	$\frac{4}{3}\sqrt{n}$	$3\,n^{1/3}$	$\frac{16}{3}n^{1/4}$
Bruck (L)	$\frac{3D}{2} \cdot \sqrt[D]{n}$	$3\sqrt{n}$	$\frac{9}{2}n^{1/3}$	$6n^{1/4}$
TRIVANCE (L)	$\frac{D}{2} \cdot \sqrt[D]{n}$	$\sqrt{n}$	$\frac{3}{2}n^{1/3}$	$2n^{1/4}$
Bucket	1	1	1	1
Swing (B)	$\frac{2^D(2^D-1)}{(2^D-2)(2^D+1)}$	1.2	1.04	1.01
TRIVANCE (B)	$\frac{3^D-1}{3^D-3}$	1.33	1.08	1.02
Rec.Doub. (B)	$\frac{2^D-1}{2^D-2}$	1.5	1.17	1.07
Bruck (B)	$3 \cdot \frac{3^D-1}{3^D-3}$	4	3.25	3.06

Table 3: Transmission delay optimalities for $D \geq 2$ torus networks. The closed-form expressions are approximated for $n \to \infty$ with respect to the ideal cost $\frac{m \cdot \beta}{D}$.

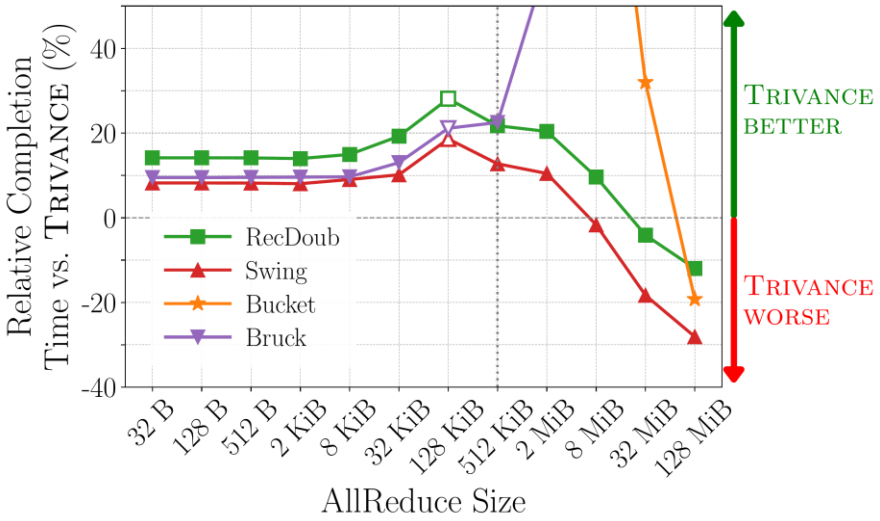
6 Evaluation

In order to complement our analytical results derived in Section 2.3 and Section 5 (in particular latency-optimality), we assess in this section the performance of TRIVANCE and state-of-the-art AllReduce algorithms empirically on different torus topologies. We conducted extensive simulations using the Structural Simulation Toolkit [16] (SST), an event-driven, packet-level network simulator. Within SST, we implemented TRIVANCE as well as Bruck's AllReduce algorithm. For the Bucket, Recursive Doubling, and Swing algorithms, we rely on the publicly available implementations by De Sensi et al. [29].

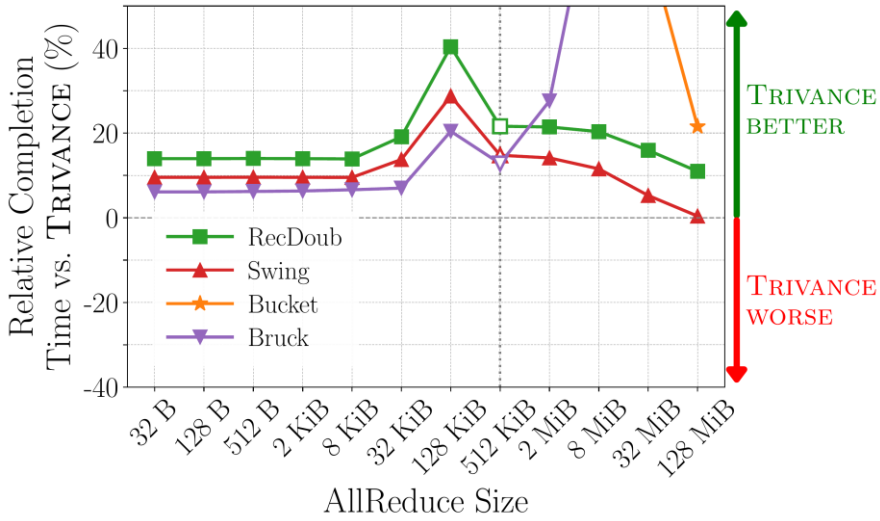
Simulation in SST

Simulator: SST (event-driven, packet-level) · 200 - 3200 Gb/s links · 200 ns per-hop latency · α = 1.5 µs/step · Message sizes: 32B – 128MiB

2D torus, 32 × 32



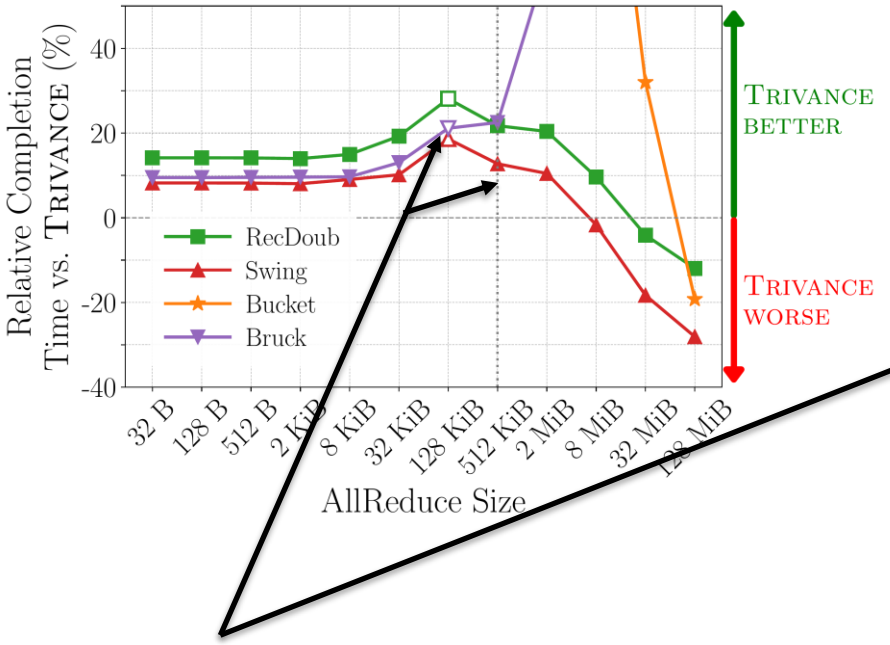
3D torus, 16 × 16 × 16



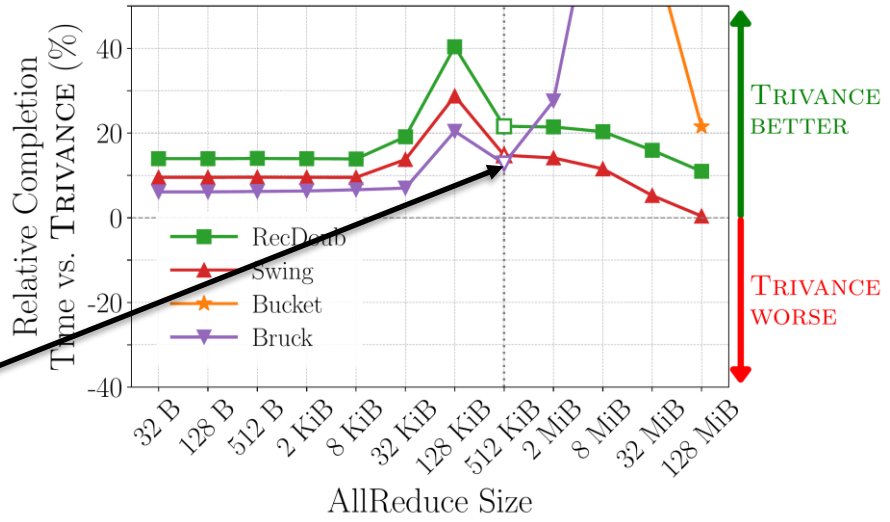
Simulation in SST

Simulator: SST (event-driven, packet-level) · 200 - 3200 Gb/s links · 200 ns per-hop latency · α = 1.5 µs/step · Message sizes: 32B – 128MiB

2D torus, 32 × 32



3D torus, 16 × 16 × 16

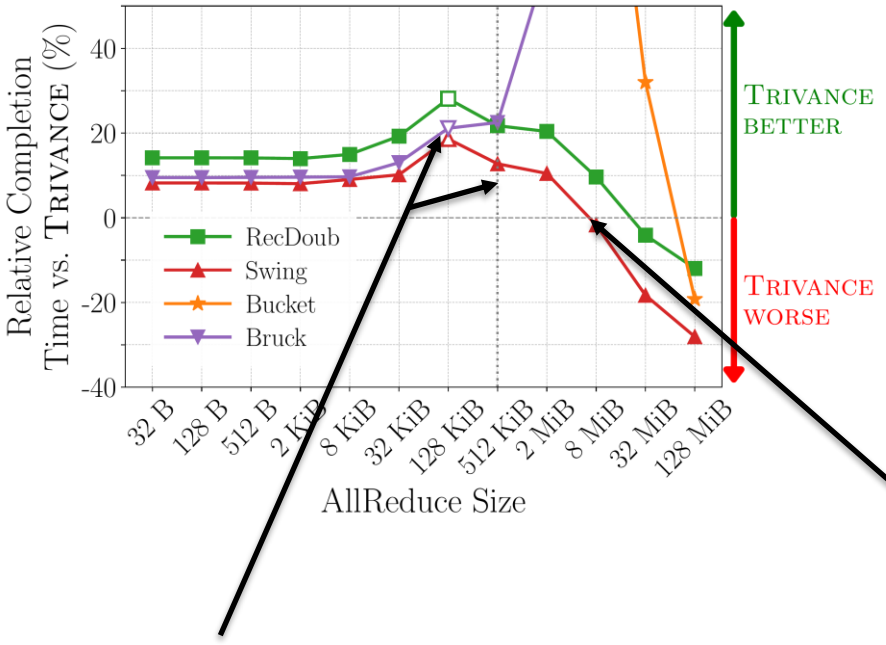


Switch from latency-optimal to bandwidth-optimal variants

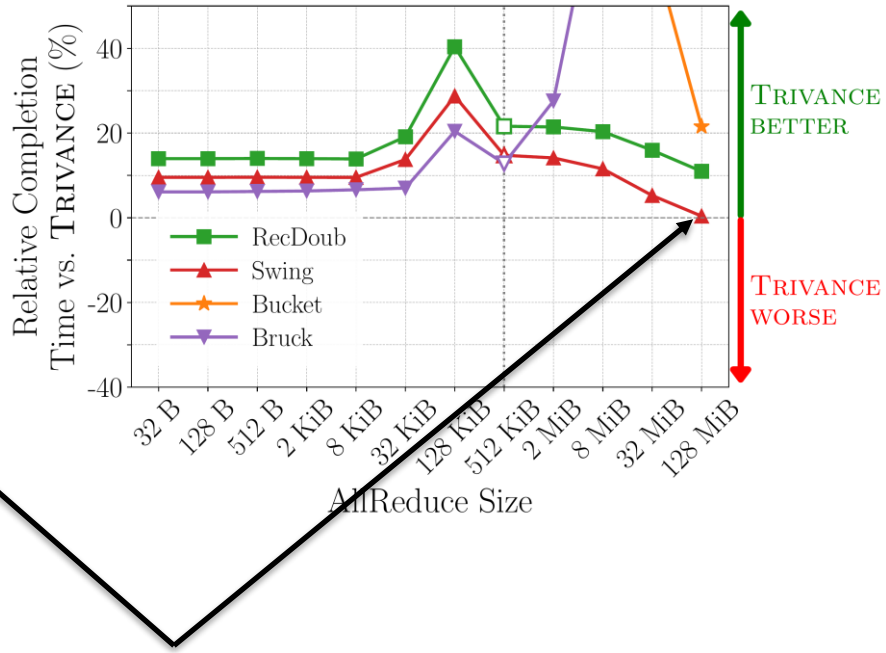
Simulation in SST

Simulator: SST (event-driven, packet-level) · 200 - 3200 Gb/s links · 200 ns per-hop latency · α = 1.5 µs/step · Message sizes: 32B – 128MiB

2D torus, 32 × 32



3D torus, 16 × 16 × 16

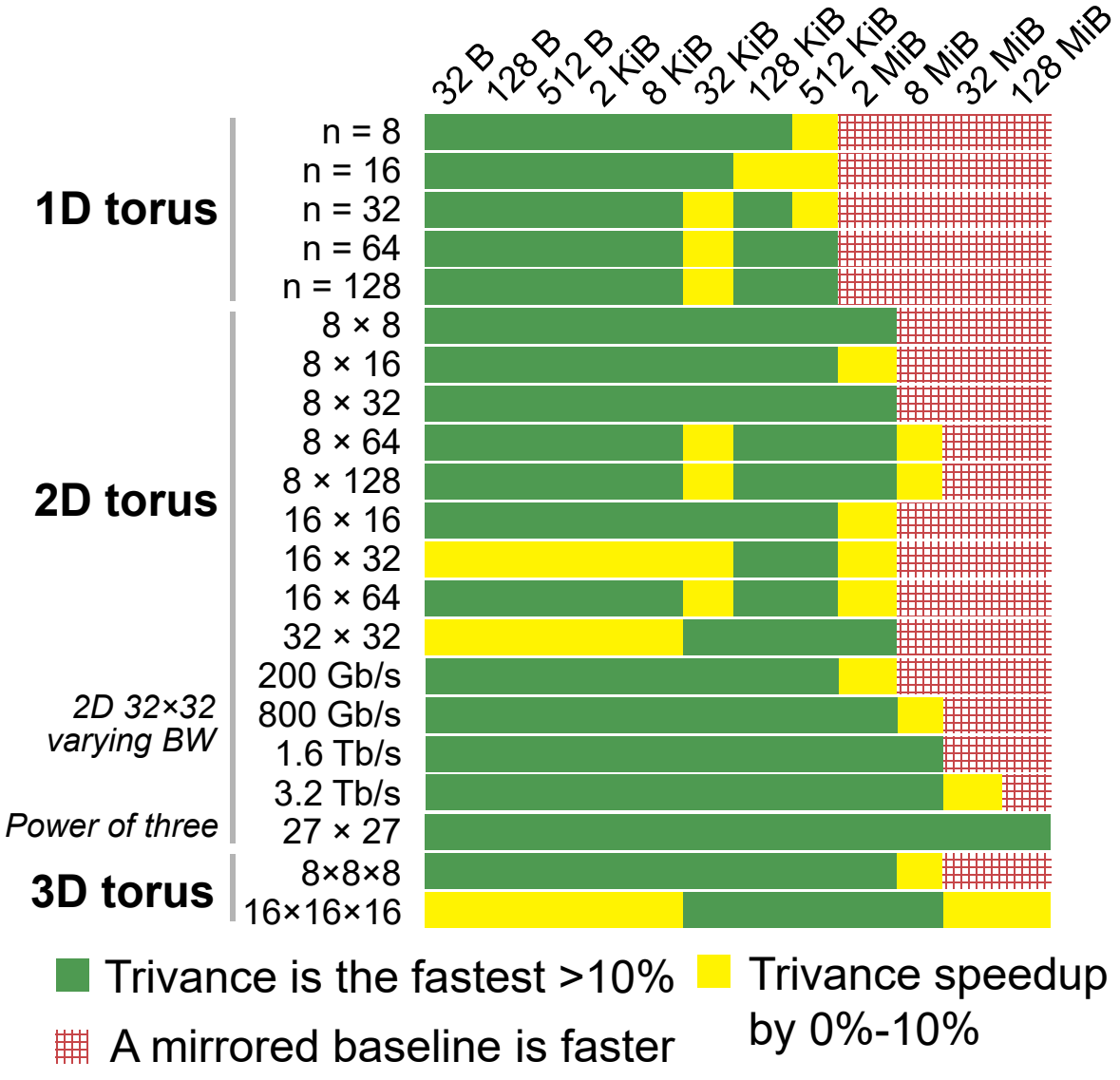


Switch from latency-optimal to bandwidth-optimal variants

Tradeoff point, where existing baselines are better due to fewer overlapping communications

When is Trivance beneficial?

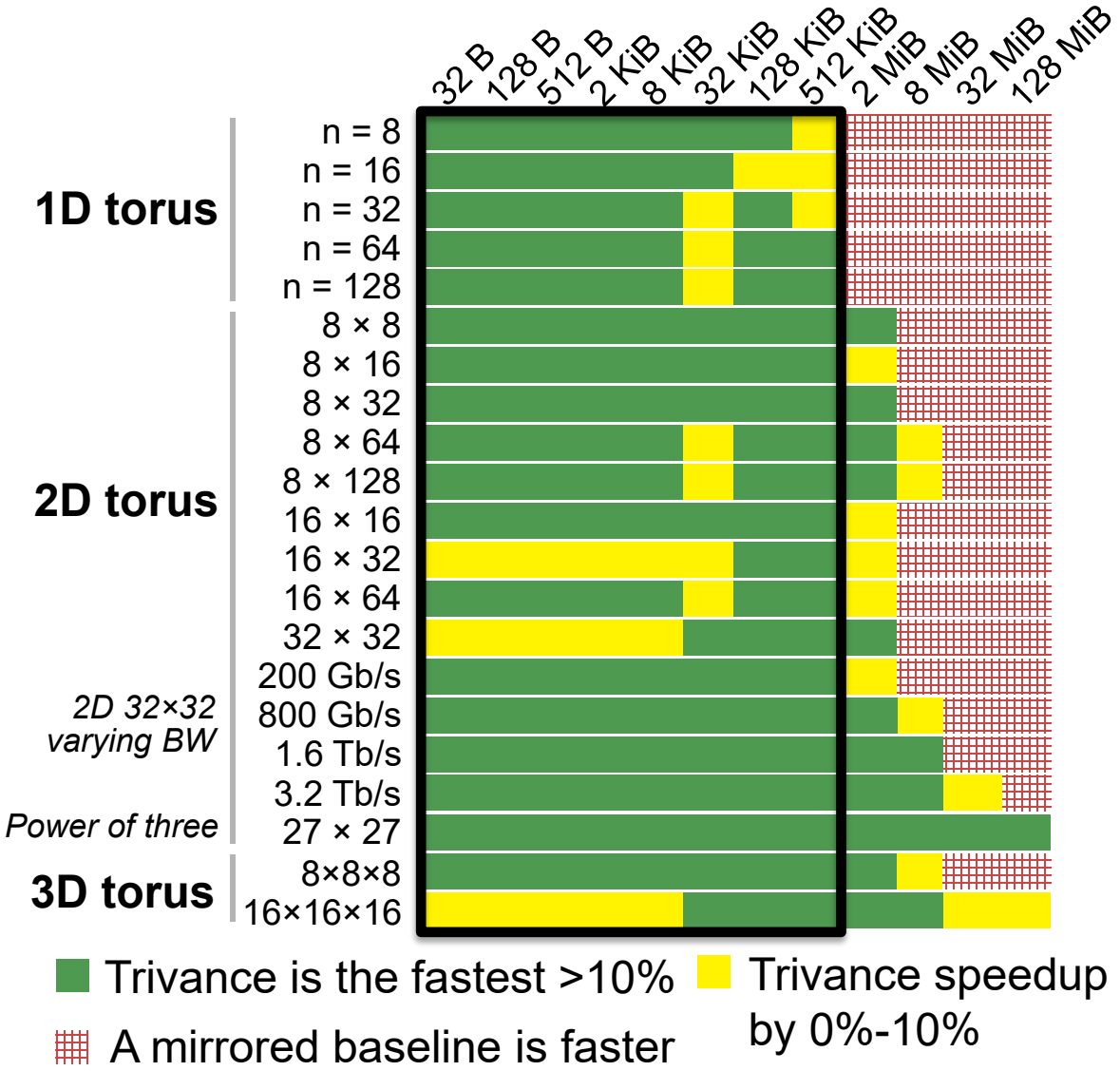
Trivance (joint-reductions) vs. baselines with mirroring



When is Trivance beneficial?

Small AllReduce sizes

Trivance (joint-reductions) vs. baselines with mirroring

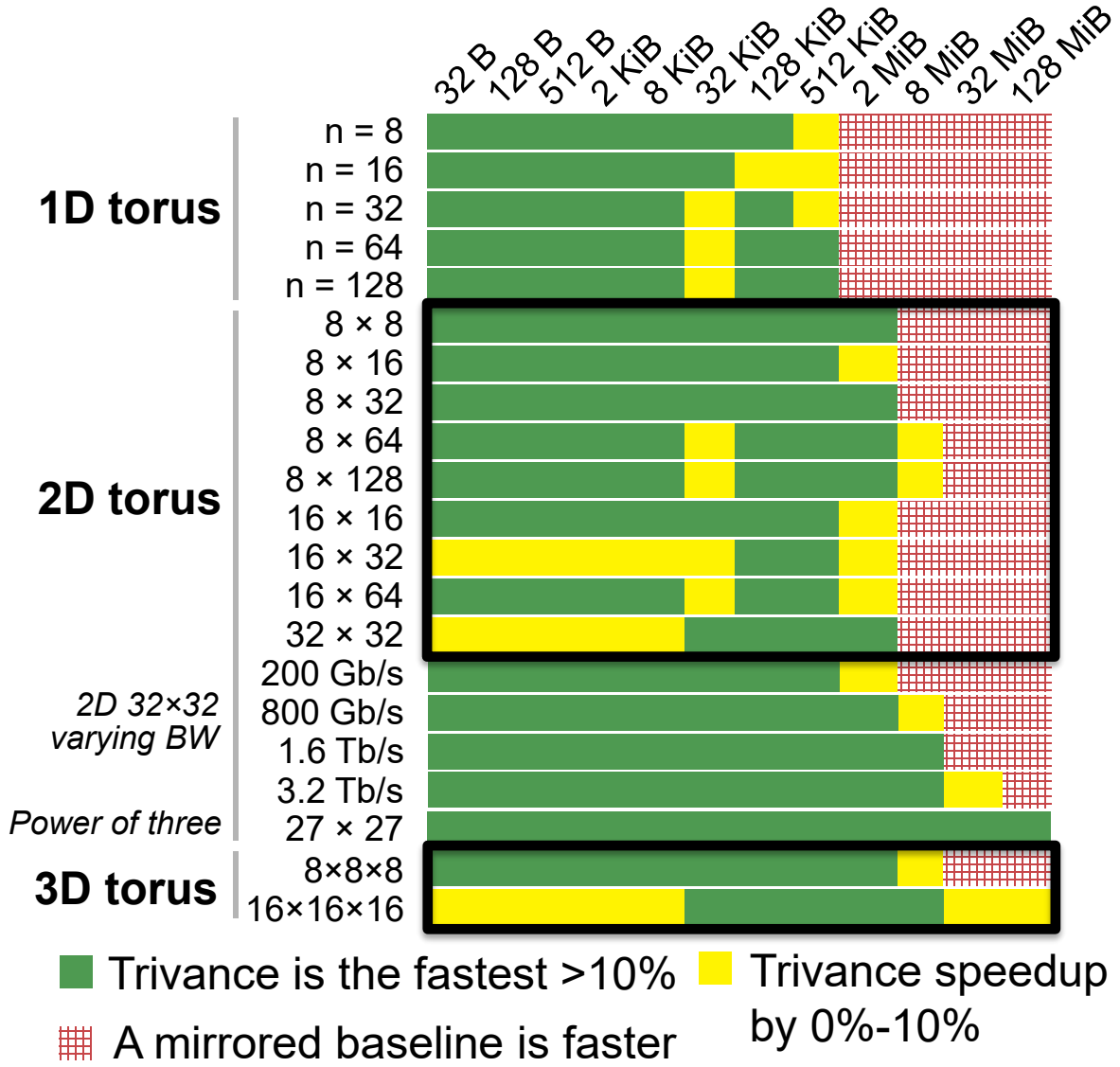


When is Trivance beneficial?

Small AllReduce sizes

Increasing dimensionality (3D or 4D tori)

Trivance (joint-reductions) vs. baselines with mirroring



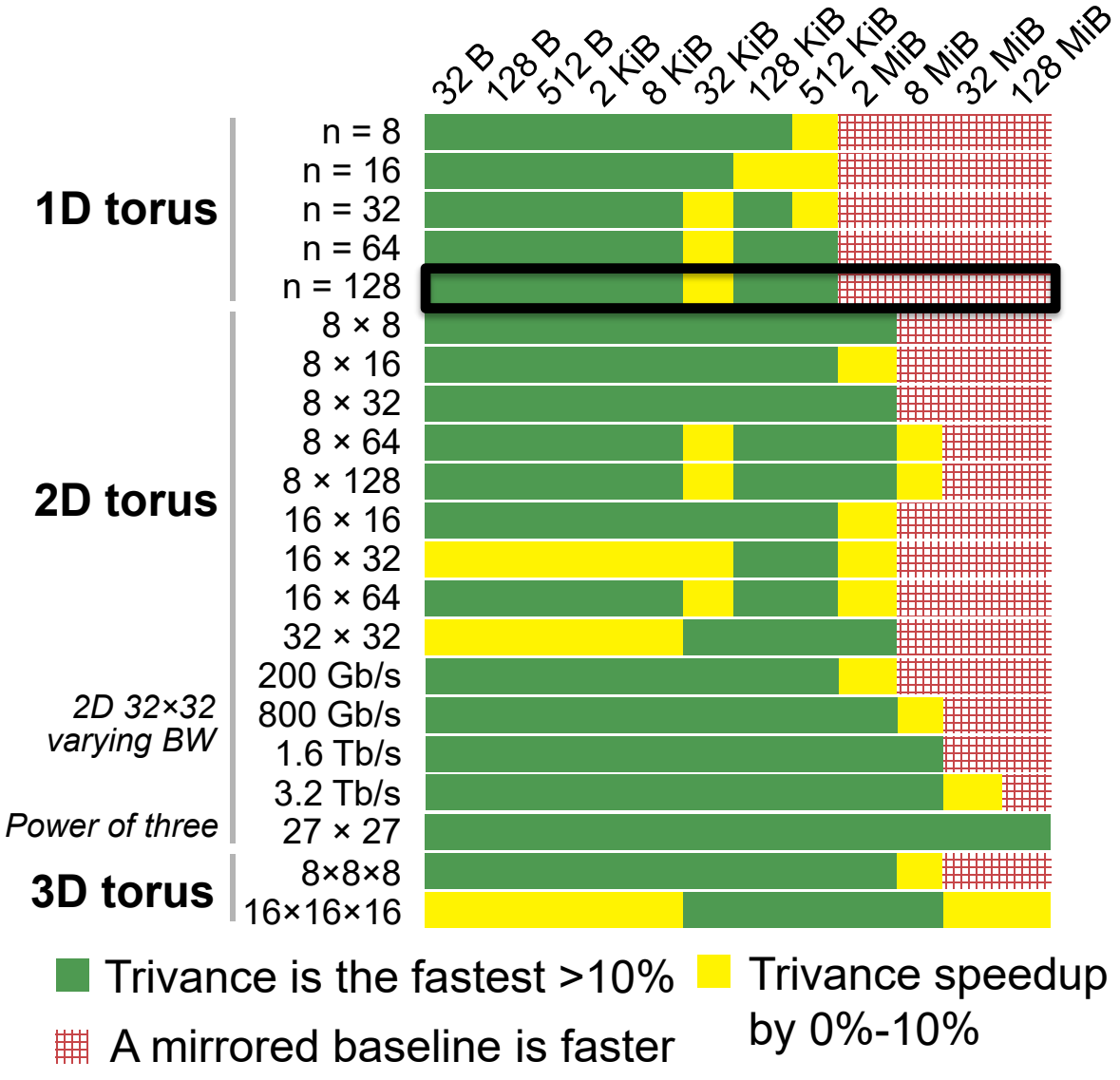
When is Trivance beneficial?

Small AllReduce sizes

Increasing dimensionality (3D or 4D tori)

Large networks/dimension sizes

Trivance (joint-reductions) vs. baselines with mirroring



When is Trivance beneficial?

Small AllReduce sizes

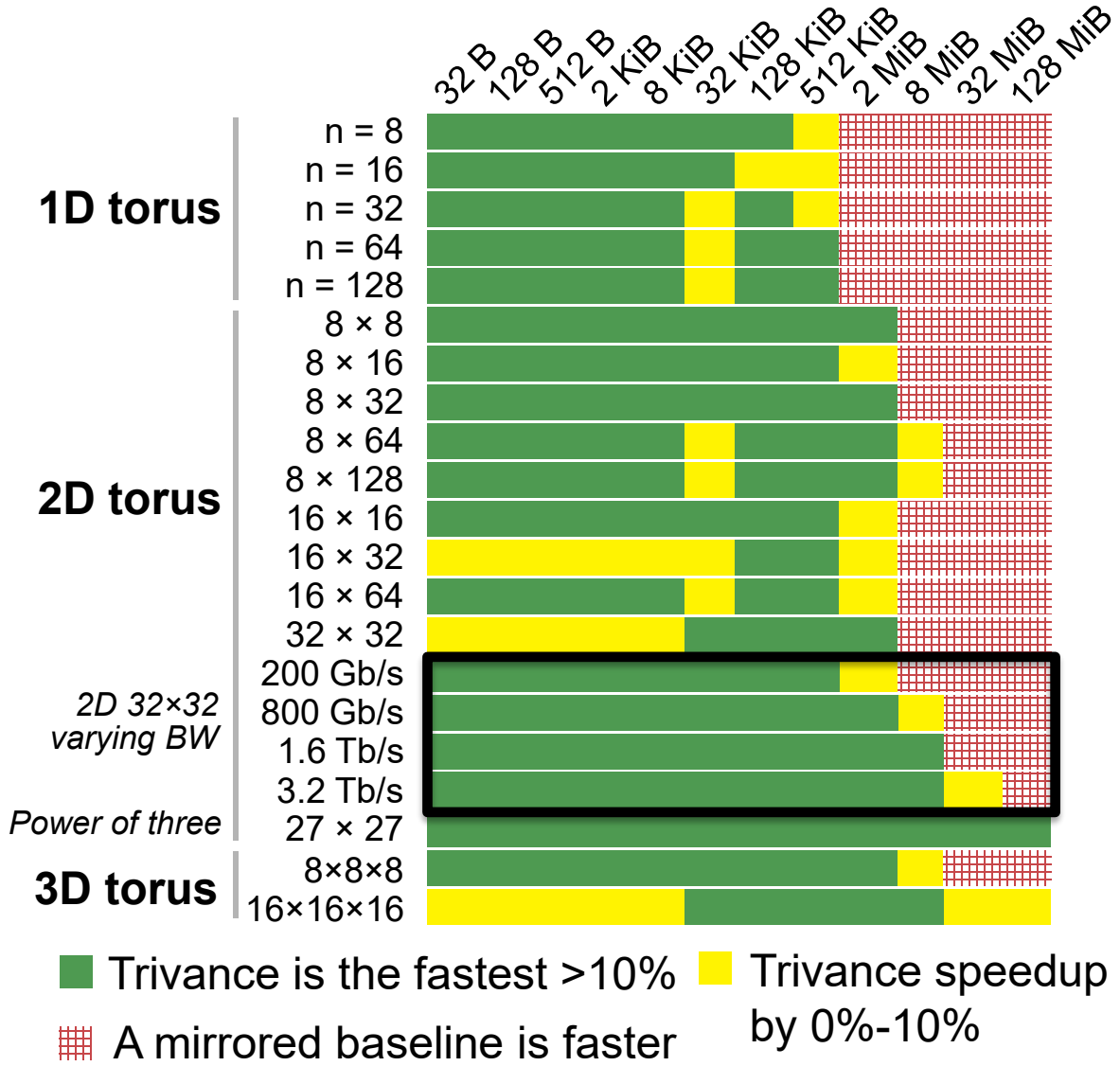
Increasing dimensionality (3D or 4D tori)

Large networks/dimension sizes

High bandwidth

High per-step or per-hop latency

Trivance (joint-reductions) vs. baselines with mirroring



When is Trivance beneficial?

Small AllReduce sizes

Increasing dimensionality (3D or 4D tori)

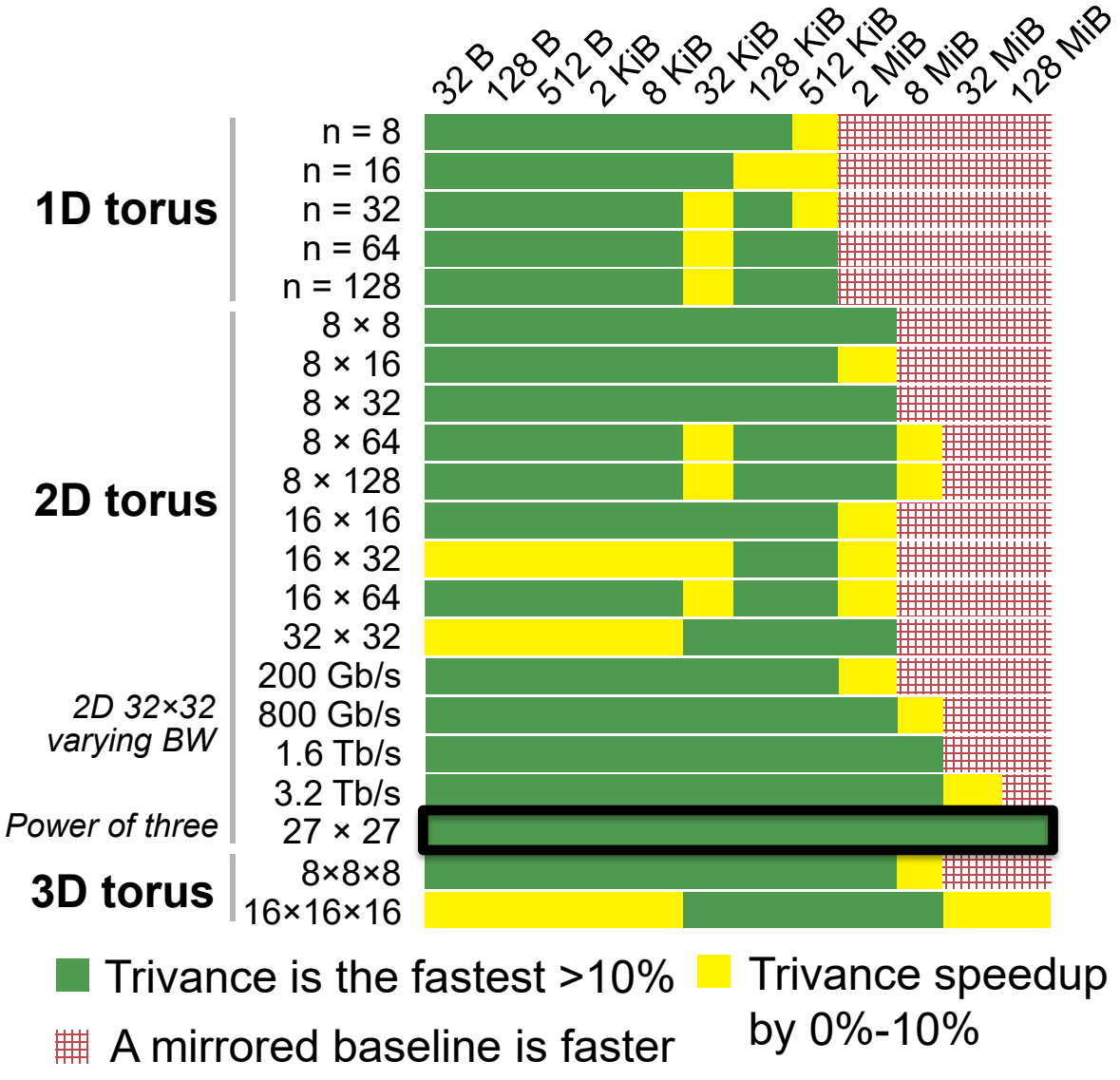
Large networks/dimension sizes

High bandwidth

High per-step or per-hop latency

Power-of-three networks

Trivance (joint-reductions) vs. baselines with mirroring



When is Trivance beneficial?

Small AllReduce sizes

Increasing dimensionality (3D or 4D tori)

Large networks/dimension sizes

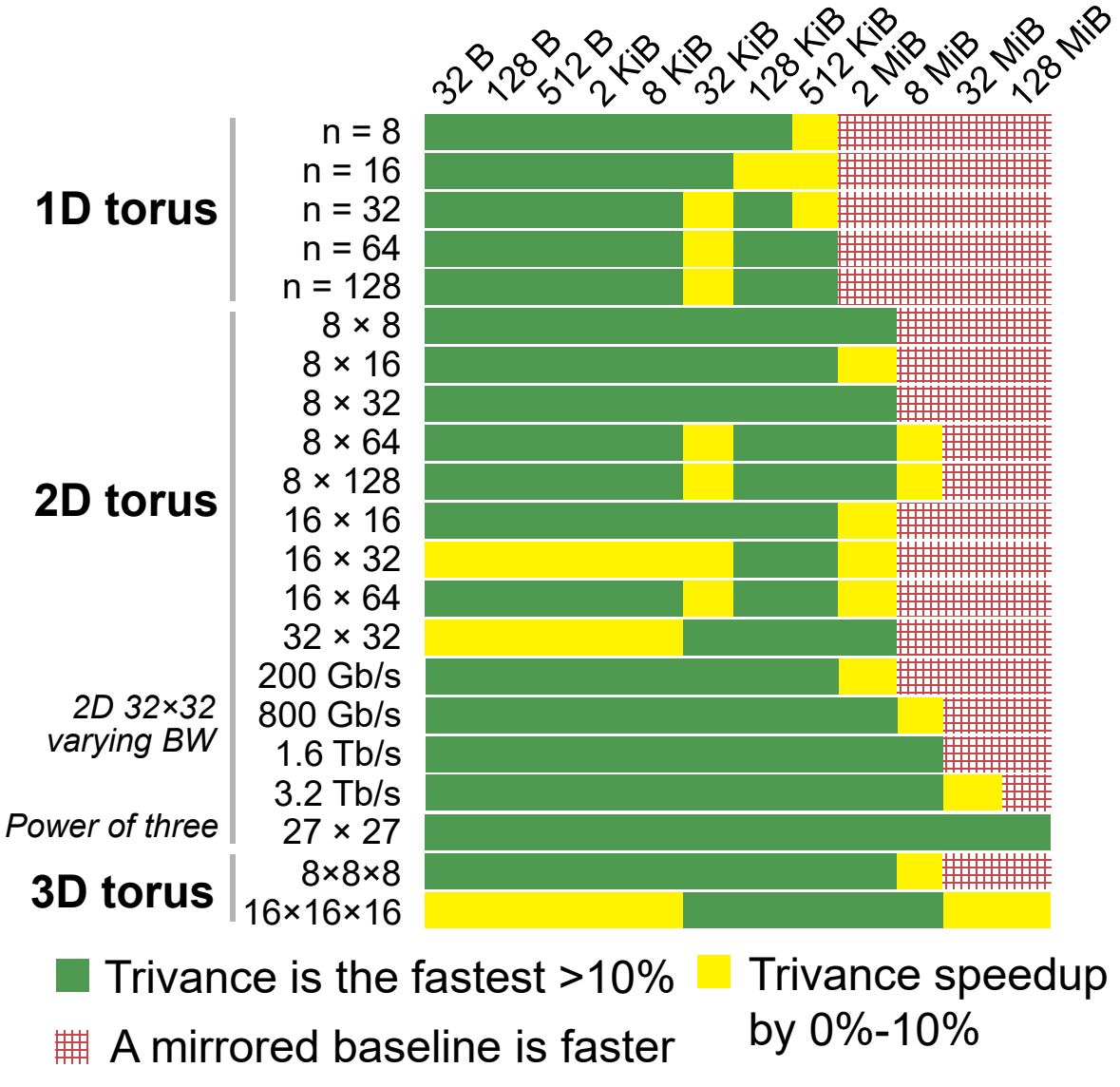
High bandwidth

High per-step or per-hop latency

Power-of-three networks

In latency-sensitive scenarios

Trivance (joint-reductions) vs. baselines with mirroring



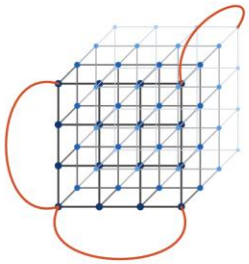
Conclusion

AllReduce and torus networks

AllReduce = Aggregation across all nodes — distribute result to all

Small AllReduce operations are increasingly latency-sensitive

Torus = mesh/grid with wrap-around links
— every dimension is a ring



— Neighbor links ⌒ Wrap-around links



Google TPU Pods
up to 4,096 chips on a 3D torus (TPUv4);
TPU 8t scales a superpod to 9,600

AWS Trainium2 UltraServer
64 chips on a 4×4×4 3D torus

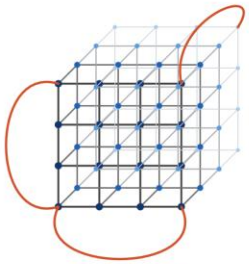
Conclusion

AllReduce and torus networks

AllReduce = Aggregation across all nodes — distribute result to all

Small AllReduce operations are increasingly latency-sensitive

Torus = mesh/grid with wrap-around links
— every dimension is a ring



— Neighbor links Wrap-around links



Google TPU Pods
up to 4,096 chips on a 3D torus (TPUv4);
TPU 8t scales a superpod to 9,600

AWS Trainium2 UltraServer
64 chips on a 4×4×4 3D torus

Trivance

↔ **Use BOTH ports simultaneously**
Communicate left and right with the same distance offset

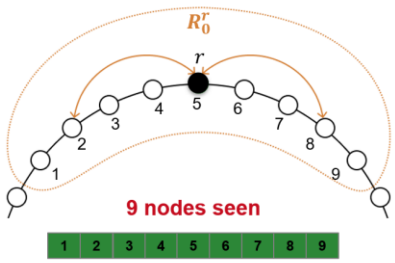
⊕ **Joint reduction**
Both incoming messages are reduced together

×3 **Triple communication distance**
Distance equals 3^k per step k

∧ **Latency-optimal**
Trivance reaches all nodes in $\log_3 n$ steps

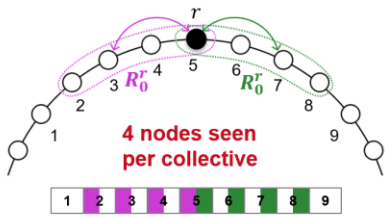
After Step $k = 1$

Trivance



9 nodes seen

Recursive Doubling



4 nodes seen per collective

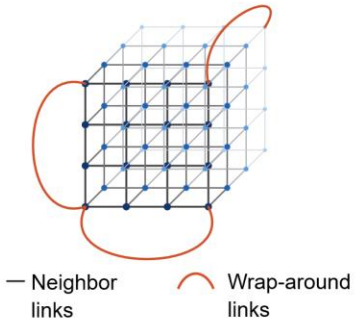
Conclusion

AllReduce and torus networks

AllReduce = Aggregation across all nodes — distribute result to all

Small AllReduce operations are increasingly latency-sensitive

Torus = mesh/grid with wrap-around links
— every dimension is a ring





Google TPU Pods
up to 4,096 chips on a 3D torus (TPUv4);
TPU 8t scales a superpod to 9,600

AWS Trainium2 UltraServer
64 chips on a 4×4×4 3D torus

Trivance

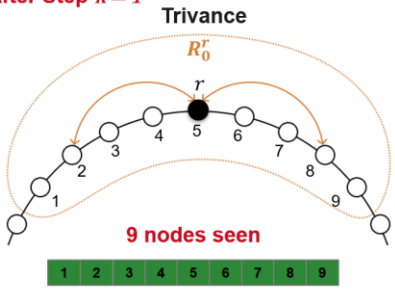
↔ **Use BOTH ports simultaneously**
Communicate left and right with the same distance offset

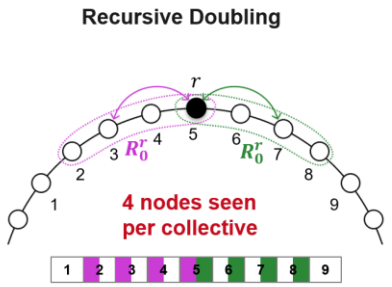
⊕ **Joint reduction**
Both incoming messages are reduced together

×3 **Triple communication distance**
Distance equals 3^k per step k

∧ **Latency-optimal**
Trivance reaches all nodes in $\log_3 n$ steps

After Step $k = 1$





Tradeoffs and optimality factors

	Ring	Rec.Doub.	Swing	Trivance
Communication steps	2n	$\log_2 n$	$\log_2 n$	$\log_3 n$
Number of hops	2n	n	$\frac{2n}{3}$	$\frac{n}{2}$
Transmission delay *Transmitted data size times number of overlapping links*	2m	$m\log_2 n$	$\frac{2m}{3}\log_2 n$	$\frac{4m}{3}\log_3 n$
Transmission delay in a 3D torus	2m	2.34m	2.08m	2.16m

Optimal | Near-Optimal | Average | Worst

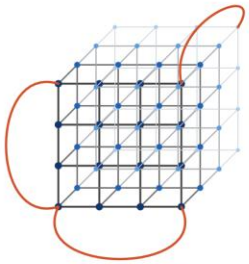
Conclusion

AllReduce and torus networks

AllReduce = Aggregation across all nodes — distribute result to all

Small AllReduce operations are increasingly latency-sensitive

Torus = mesh/grid with wrap-around links
— every dimension is a ring



— Neighbor links Wrap-around links



Google TPU Pods
up to 4,096 chips on a 3D torus (TPUv4);
TPU 8t scales a superpod to 9,600

AWS Trainium2 UltraServer
64 chips on a 4×4×4 3D torus

Trivance

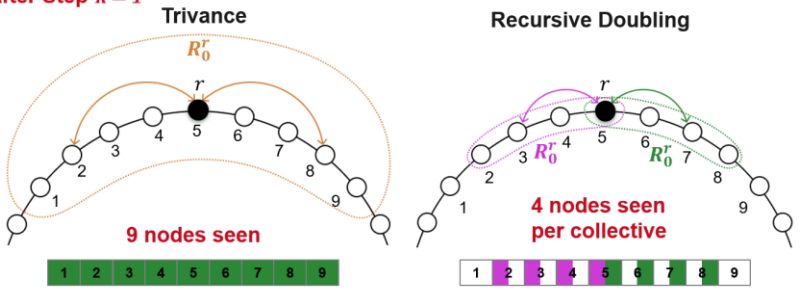
↔ **Use BOTH ports simultaneously**
Communicate left and right with the same distance offset

⊕ **Joint reduction**
Both incoming messages are reduced together

×3 **Triple communication distance**
Distance equals 3^k per step k

∧ **Latency-optimal**
Trivance reaches all nodes in $\log_3 n$ steps

After Step $k = 1$



Tradeoffs and optimality factors

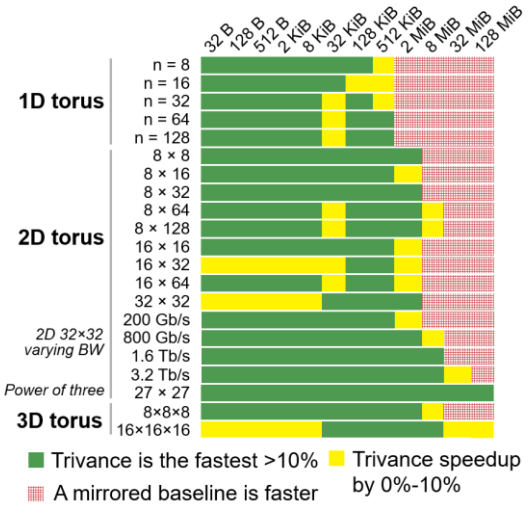
	Ring	Rec.Doub.	Swing	Trivance
Communication steps	2n	$\log_2 n$	$\log_2 n$	$\log_3 n$
Number of hops	2n	n	$\frac{2n}{3}$	$\frac{n}{2}$
Transmission delay	2m	$m\log_2 n$	$\frac{2m}{3}\log_2 n$	$\frac{4m}{3}\log_3 n$
Transmitted data size times number of overlapping links				
Transmission delay in a 3D torus	2m	2.34m	2.08m	2.16m

Optimal Near-Optimal Average Worst

When is Trivance beneficial?

Small AllReduce sizes
Increasing dimensionality (3D or 4D tori)
Large networks/dimension sizes
High bandwidth
High per-step or per-hop latency
Power-of-three networks

In latency-sensitive scenarios



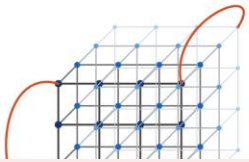
Conclusion

AllReduce and torus networks

AllReduce = Aggregation across all nodes — distribute result to all

Small AllReduce operations are increasingly latency-sensitive

Torus = mesh/grid with wrap-around links
— every dimension is a ring





Trivance

Use BOTH ports simultaneously
Communicate left and right with the same distance offset

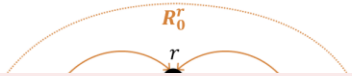
Joint reduction
Both incoming messages are reduced together

Triple communication distance
Distance equals 3^k per step k

Latency-optimal
Trivance reaches all nodes in $\log_3 n$ steps

After Step $k = 1$

Trivance



Recursive Doubling



Thank you!

Email: juerss@campus.tu-berlin.de

Number of hops	2n	n	$\frac{2n}{3}$	$\frac{n}{2}$
Transmission delay	2m	$m\log_2 n$	$\frac{2m}{3}\log_2 n$	$\frac{4m}{3}\log_3 n$
Transmitted data size times number of overlapping links				
Transmission delay in a 3D torus	2m	2.34m	2.08m	2.16m

Optimal · Near-Optimal · Average · Worst

or 4D tori)

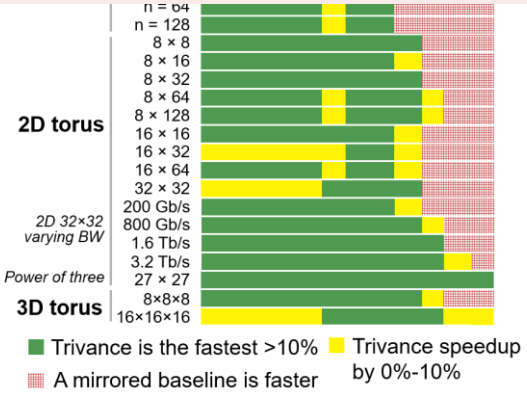
Large networks/dimension sizes

High bandwidth

High per-step or per-hop latency

Power-of-three networks

In latency-sensitive scenarios



Thank you!

Email: juerss@campus.tu-berlin.de

AllReduce on rings

Cost function:

$$T(m, A) = \underbrace{\sigma(A) \cdot \alpha}_{\text{Latency}} + \underbrace{\sum_{k=0}^{\sigma(A)-1} m_k \cdot c_k \cdot \beta}_{\text{Transmission Delay}}$$

$\sigma(A) =$ number of communication steps

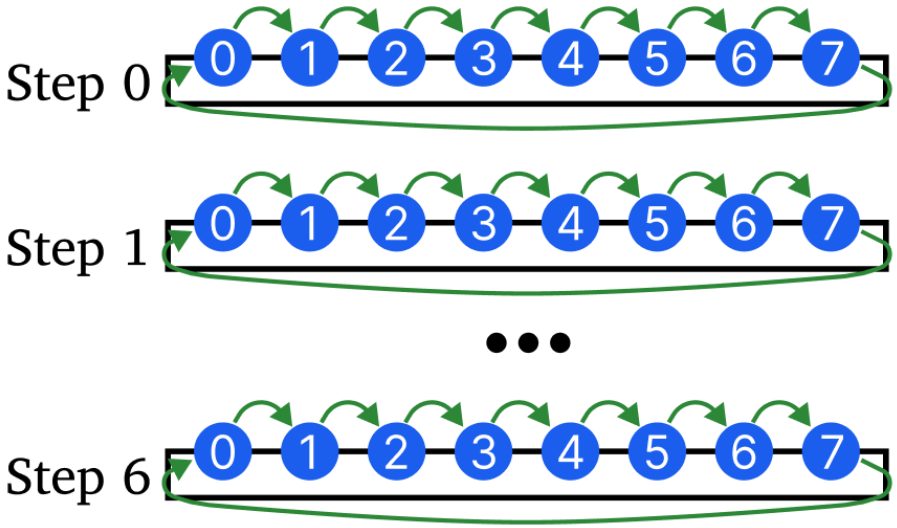
$\alpha = latency\ cost$

$\beta = bandwidth\ cost$

$m_k = message\ size$

$c_k = network\ congestion$

Hamiltonian Ring AllReduce



Steps: Linear $2(n-1)$ — slow for large n

Congestion: None, no overlap

Path length: Linear $n-1$

AllReduce on rings

Cost function:

$$T(m, A) = \underbrace{\sigma(A) \cdot \alpha}_{\text{Latency}} + \underbrace{\sum_{k=0}^{\sigma(A)-1} m_k \cdot c_k \cdot \beta}_{\text{Transmission Delay}}$$

$\sigma(A) =$ number of communication steps

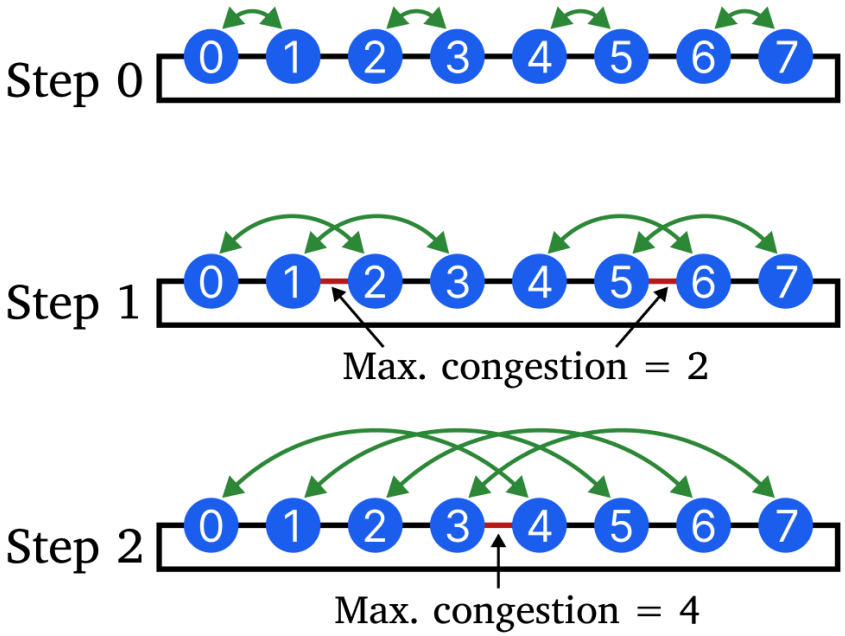
$\alpha = latency\ cost$

$\beta = bandwidth\ cost$

$m_k = message\ size$

$c_k = network\ congestion$

Recursive Doubling AllReduce



Steps: $\log_2 n$ steps

Congestion: 2^k overlaps per step k

Path length: Linear $n - 1$

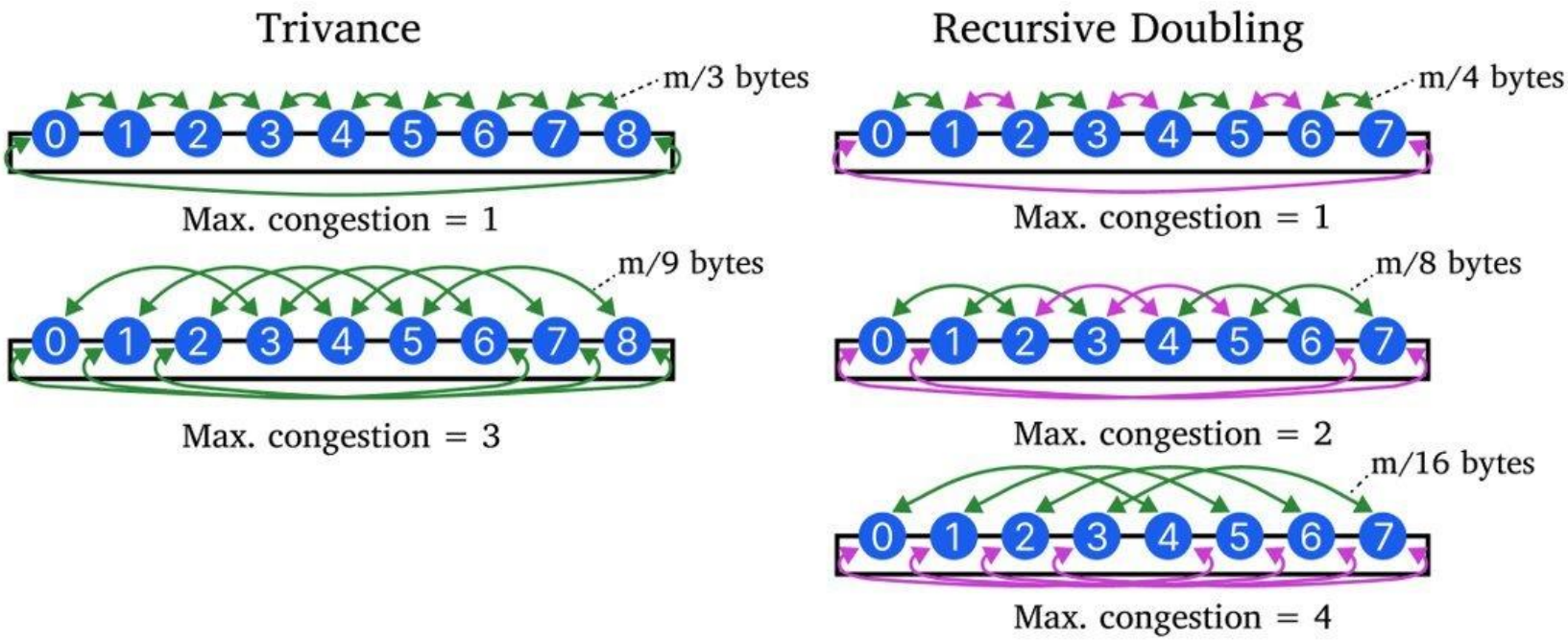
Trivance

Use BOTH ports simultaneously
Communicate left and right with the same distance offset

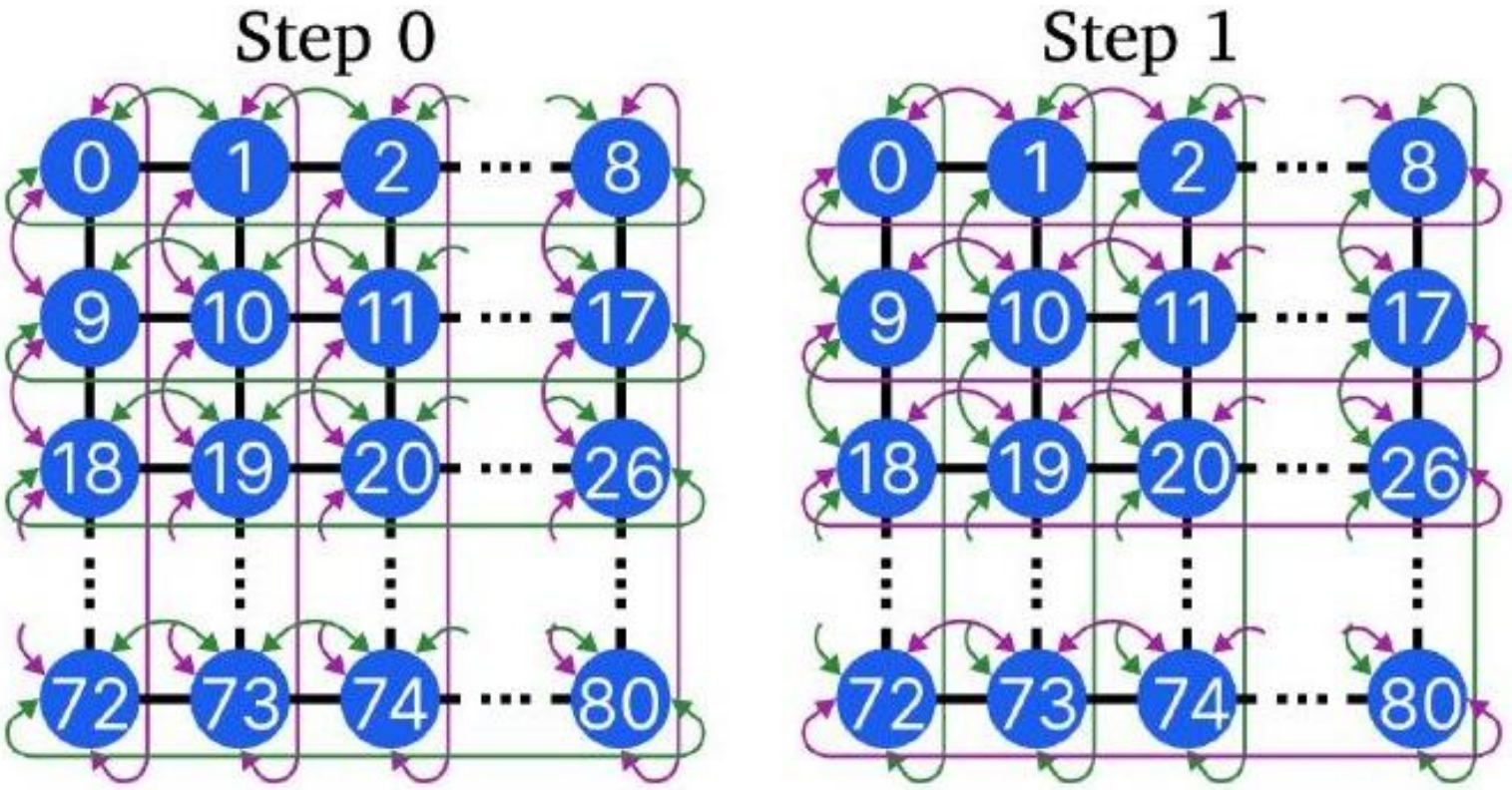
Triple communication distance
Distance grows as 3^k per step

Joint reduction
Both incoming messages are reduced together before forwarding

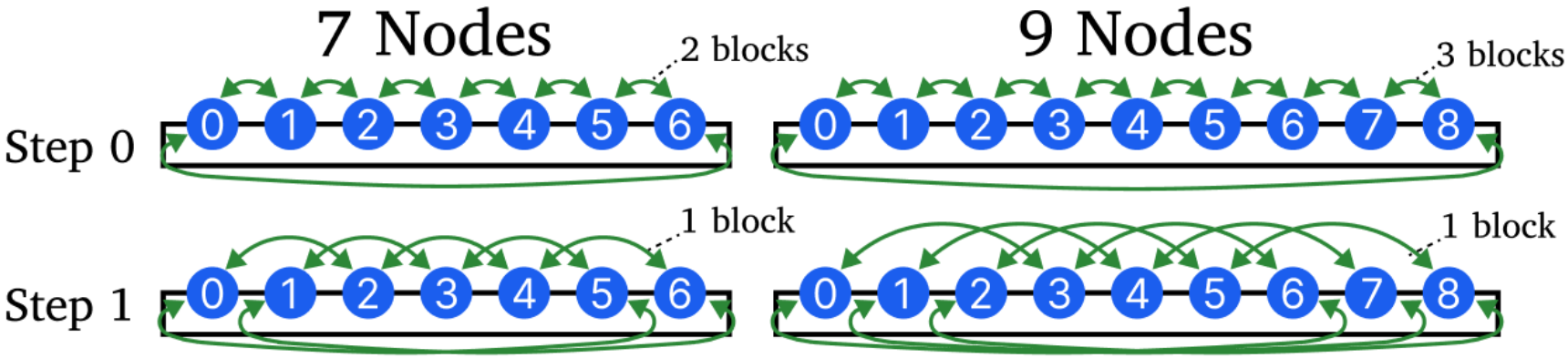
Latency-optimal
Trivance reaches all nodes in log_3 n steps



Generalization to multidimensional tori



Extention to arbitrary network sizes



Trivance

Ring Optimalities for Latency, Bandwidth and Tx

Algorithm	La. Opt.(Λ)	Bw. Opt.(Δ)	Tx. Opt.(Θ)
Ring	$\frac{2n}{log_3 n}$	1	1
Rec.Doub. (B)	$2 \cdot \log_2 3$	1	$\frac{1}{2} \log_2 n$
Swing (B)	$2 \cdot \log_2 3$	1	$\frac{1}{3} \log_2 n$
Bruck (B)	2	1	$2 \log_3 n$
TRIVANCE (B)	2	1	$\frac{2}{3} \log_3 n$
Rec.Doub. (L)	$\log_2 3$	$\frac{\log_2 n}{2}$	n
Swing (L)	$\log_2 3$	$\frac{\log_2 n}{2}$	$\frac{n}{3}$
Bruck (L)	1	$\log_3 n$	$\frac{3n}{2}$
TRIVANCE (L)	1	$\log_3 n$	$\frac{n}{2}$

Torus Optimalities for Transmission Delay

Algorithm	Closed-form	D=2	D=3	D=4
Rec.Doub. (L)	$D^2 \sqrt[D]{n}$	$4\sqrt{n}$	$6n^{1/3}$	$8n^{1/4}$
Swing (L)	$\frac{D^2}{3} \sqrt[D]{n}$	$\frac{4}{3}\sqrt{n}$	$3\,n^{1/3}$	$\frac{16}{3}\,n^{1/4}$
Bruck (L)	$\frac{3D}{2} \cdot \sqrt[D]{n}$	$3\sqrt{n}$	$\frac{9}{2}n^{1/3}$	$6n^{1/4}$
TRIVANCE (L)	$\frac{D}{2} \cdot \sqrt[D]{n}$	$\sqrt{n}$	$\frac{3}{2}n^{1/3}$	$2n^{1/4}$
Bucket	1	1	1	1
Swing (B)	$\frac{2^D(2^D-1)}{(2^D-2)(2^D+1)}$	1.2	1.04	1.01
TRIVANCE (B)	$\frac{3^D-1}{3^D-3}$	1.33	1.08	1.02
Rec.Doub. (B)	$\frac{2^D-1}{2^D-2}$	1.5	1.17	1.07
Bruck (B)	$3 \cdot \frac{3^D-1}{3^D-3}$	4	3.25	3.06